

Recipe Manager & Timing Application

Contents

Analysis

Problem

- [Overview](#)
- [Stakeholders](#)
- [Stakeholder Interview](#)
- [Why it is suitable to a computational problem](#)

Research

- [Existing products](#)
- [Ways of doing the timings for recipes](#)
- [Existing tools](#)

Requirements

- [Hardware](#)
- [Software](#)

Solution

- [Limitations of a computational solution](#)
- [Essential Features](#)
- [Success criteria](#)

Design

Database design

- [Relationship entity diagram](#)

Code design

- [Class diagram](#)
- [Network generation algorithm](#)
- [Test data](#)
- [Data validation](#)
- [Data verification](#)
- [Key data structures](#)

Interface design

Development and Testing

Testing to inform development

Evaluation

Testing against success criteria

Evaluating Usability

Evaluating Features

[Maintaining the application](#)

[Final code](#)

[Python](#)

- [main.py](#)
- [userManager.py](#)
- [databaseManager.py](#)
- [utils.py](#)

[SQL](#)

[References](#)

Analysis

Problem

Overview

There is no easy way for people who have just started cooking to manage and discover new recipes in an application with the ability to load those recipes into a timer allowing them to keep on track whilst cooking. There are applications which let people share recipes and there are numerous timer apps but one which does both in an easy to use way is needed.

I want to make an application that fixes this by aiding people when cooking. It will have some sort of recipe searching/making feature which stores information about how to time the cooking process to be loaded into a timer. It needs to be intuitive and easy to use for the user so that it is practical and will help them whilst cooking rather than be confusing.

Stakeholders

The app will be designed to aid people cooking with timings and recipes. The timing feature is something that everyone can use but the recipe section is more tailored to people who haven't learnt as many recipes as someone with more experience will have had, such as people who just started cooking for themselves like university students. The main stakeholders will be people who: cook for themselves, and so don't have help already; younger people, as they are less experienced. The fact they are younger also means that they are more likely to use technology and want to use an app to aid them.

However both features, timer and recipes, would be useful to everyone. It would make finding new recipes easier and the timer is useful to everyone when cooking.

Stakeholder Interviews

Questions :

1. How much experience do you have with cooking?
2. Do you often try new recipes?
 - 2.1. If yes, what would you want from an app to help make it easier?
 - 2.2. If no, would you try new recipes if there was an easy way to do it? What would you want from an app to make it easier for you?
3. Is timing a problem for you whilst cooking?
 - 3.1. If yes, have you tried timing apps to help you? Do you still use them and why/why not?
 - 3.2. If not, might a timing app help you? What would you want from it?

4. What kind of devices do you have and which would you want the app to be on for use in the kitchen?

Interview 1:

Interviewee 1 is a student about to go into university. They have some experience with cooking and will start cooking for themselves regularly for university.

1. "I don't have any experience. My mum does all the cooking at home." "I occasionally help my mum with cooking but not very often, I will start cooking for myself often in September due to university"

2. "We try new things about once a month. Usually through a food delivery box app"

Gusto [[009 - Gusto \(04/06/2023\)](#)]

2.1. "I would like to be able to see what ingredients I have and make something easy, not having to shop for new ingredients"

3. "No it isn't, but I would like the option of making different foods depending on how much time I have as my situation is always changing with being busy"

3.2 "Yes, I would like to try and I think it would help. I would like to have a split screen so I can see multiple timers at once, that are easy to label so I don't have to remember which is which, as well as an alert for when each is done"

4. "I have a phone, iPad and computer as well as a watch. I would like it to be on my iPad so I have a bigger screen to see what I need to do"

Notes:

Some sort of timing app would be useful to them. They already use a recipe recommendation app along with a monthly box with recipes and ingredients in. A way to help them time their recipes would be useful as there isn't currently that feature in the system they use.

Interview 2:

Interviewee 2 is a student at university, they cook for themselves everyday.

1. "I have quite a bit of experience with cooking, I'm in my second year at university and I cook for myself everyday." "Before coming to university I had some experience but had to mainly learn here"

2. "Not really, it's difficult to find time to learn and cook them by myself along with university so I tend to stick to the same few simple dishes."

2.2. "Yes, I would love an easier way to have variety in my diet. I'm constantly using my phone so an app would be great. I know lots of my friends who would benefit from that kind of thing."

3. "Not really as I only cook those simple dishes but if I were to cook more complex dishes I think it would be, as I would be trying to learn and figure out how to cook the dish as well as cook it."

3. 1/2. "I haven't tried timing apps before, I've just used the built in timer in my phone. I think it would be useful though if I were to cook more complex recipes."

4. "I don't have anything but my phone so an app on their would be ideal"

Notes:

A timing application on their phone would be ideal. They haven't cooked many complex recipes but would like to so a recipe browsing system along with the timer system would allow them to discover the new recipes and give them the help they need to cook them.

Interview 3:

Interviewee 3 is a parent, they cook for their family everyday. They are also interested in dieting and have done research into it and used services before to help.

1. "I have got lots of experience with cooking, I have been doing it for many years and cooked a wide variety of recipes"

2. "Around once a week, I sometimes get food delivery boxes with recipients and ingredients and sometimes ones with just spices and ingredients. I also use a recipe recommendation app."

2.1. "This app has lots of nutritional information about food and recipes. It gives things health scores which represent an overview of how healthy or unhealthy a meal is. It also can do this for meals you have eaten, provided a list of ingredients. It's recipe browsing and recommendation systems are also very good, having the option of filters to find recipes of a certain type, you can also view seasonal recipes. It has instructions for making the recipe, however it's consistent and doesn't have any aid when making the recipe like built-in timers, that would be something I would find useful. It has the ability to put a score against it to say how much I liked the recipe and if would want to cook it again. It also give you the option to pull recipes from the web given a link or from an image, this is really useful and still gives you the option to go back to the source with the link.

3. "I don't find it a problem when cooking, I have my phone and a timer built into the oven. I don't usually find this system a problem but a built-in system would be useful for new and more difficult recipes."

3.2. "Some way of searching for a recipe given how much time you have to make it, also something similar with what ingredients you have would be good."

4. "I have a phone, it's the easy use in the kitchen as it doesn't get in the way as it's small. Also it can be moved around to wherever you are working easily."

Dietitian:

Questions:

1. What would you want from a recipe app which you can give to your clients to help with their eating?

2. How would you like the ability to add recipes be implemented?

Interview 1:

Interviewee 1 is a dietitian who helps clients with eating more healthily and to work around certain health conditions.

1. "I would want your app to calculate nutritional information for each recipe. It should also allow the user to be able to sort by that nutritional info, for example someone who is diabetic should be able to sort by low carbohydrate and high fibre foods. I want it to be able to also tell the user if it is good for certain diets or health conditions, e.g. low carbs, high fibre recommendations for someone who is diabetic. And also more generally I think there should be ways of searching for more healthy recipes. Less red meat, whole grain produce, more fish, healthy fats, e.t.c."

2. "I would like the app to calculate nutritional information or have ways of displaying it on the app. I would also like a way of verifying the source of the recipe, for example a dietitian should be able to show to be a trusted source of recipes. I would also like a way of making recipes private only accessible through a link of if a user has saved it in order to share my own paid recipes with your app. This would also be useful for cookbooks and any other similar things."

Why it is suitable to a computational solution

The timer aspect of the application is something which will have an output for any valid input, the input is given by the user in the form of making a recipe and there is an unlimited amount of recipes. The recipe making feature will be designed in a way to allow the user to make any recipe in the app. There will also be an output (the timer) for any valid recipe input making the solution solvable with a computational method.

This calls for a computational solution as it isn't feasible to work out the timings for each recipe a user makes, as it would take many times longer than a computational solution. A computational solution would generate the timings in an appropriate amount of time, meaning the solution would be tractable. It would also allow for features not possible with non-computational methods, such as videos.

It's also appropriate for a computational solution as the recipes can be stored in networks which have been designed already and many algorithms discover to work on them which would make the design of that system a lot easier and more efficient and reliable. These system include gantt charts and weighted network with various algorithms which work with them such as A* and dijkstra's algorithm [[found here. under \(Ways of doing the timings for recipes\)](#)].

The problem is clearly defined, as shown in the [success criteria](#) and so I will be able to decompose it during design. This will also allow for an object oriented approach making code readability and design easier.

The solution uses a phone which is a device people are familiar with and comfortable using, this will make the solution more intuitive and accessible to people.

Research

Existing Products

Whisk [[001 - Whisk \(23/03/2023\)](#)]

Whisk is a recipe sharing application. It allows users to create, share and find recipes. Each recipe has information about how to cook it, ingredients and estimated preparation and cooking times. Recipes are recommended to you via a personalised feed as well as through 'communities' which are themed collections of recipes

You can log in with google or other pre-existing accounts which makes it easy for users to sign up, however there is no way of accessing it without signing up. Giving users the option to get a recipe or making a one time timer without making an account is useful as not everyone will need the features which require an account such as recommendations. The logging in process asks lots of questions which takes some time which some users will not be bothered to do, however some of it is very good. It uses info on dietary restrictions, food preferences and more to recommend food to the user. Communities and creators are options to join/follow when starting an account, however there is very little information given on what happens if you join them and what each one is. Some option to browse them later on being suggested after you finish setting up your account would be better especially if there is some information on what they are.

There are many options to search for recipes such as: ingredients, meal type (breakfast, lunch, snack, drinks, e.t.c.), diet, timing, nutrition and more. This is very useful and intuitive. The communities and creators allow for you to search for recipes as well but feel too forced in the account setup and less intuitive.

The recipes have 3 tabs: "Ingredients", "Instructions" and "Health score" although the names don't fully relate to the content. Ingredients has information on: a brief description, sometimes prep/cooking times, ingredients needed with quantities (and an option to change servings and calculate new ingredient quantities required), ratings/tags and comments. The content is useful on this tab but better structure would make it more intuitive for new users trying to find things. The Instructions tab is just a website embedded into the app, this isn't good as it leads to lots of ads, problems with pages in different languages and links to different places. Also as it is pulling from a website there is no standard format making it harder to read/navigate. The health score tab is what it says, it has information about its health score for various categories (calories, sugars, protein, e.t.c.) and an overall score.

In conclusion the application is good, it has a very useful recommendation algorithm as well as intuitive search options, a good search page would be good for my app. Recipes are easy to understand at a glance, having a nice UI. I like the option to change serving sizes to get different ingredient quantities and the comments. I dislike the web embed for the instructions and there is no timer functionality in the application.

Kitchen Multi-Timer [[002 - Kitchen Multi-Timer \(23/03/2023\)](#)]

Kitchen Multi-Timer is a timer application. It allows the user to create multiple timers. There are a few temporary timers which you set every time you use the app and a page of preset timers which you can save to use multiple times.

Each timer is standalone, not affecting other timers and so has no aid past timing things

(such as organising what order things have to be done in). Making timers is easy to do, each one being able to be named, It has an intuitive ui and is easy to use.

There is no way of signing up but as it has no recipe features it is appropriate. It does keep asking you to rate the app even in the first minute of use when you haven;t had a chance to use the app really.

In conclusion the app is nice and simple, an intuitive ui would be good for my app as well as ways of having pre-made timers if a full recipe wasn't wanted.

Cooking Timer: scheduler [[003 - Cooking Timer: scheduler \(23/03/2023\)](#)]

Cooking timer: scheduler is a timer application. It allows the users to make 'meals' which are collections of timers set up to help you time your meal. There is no option to use it without doing a 'meal' and so one time timers aren't easy to do.

Each timer is made up of ingredients of a set length and have to start at or finish at a set time in relation to the start of the cooking process. This makes structuring a recipe quite good but the ingredients are separate from each other and so if one takes longer than expected the recipe doesn't adjust accordingly. It automatically calculates when everything happens from a start or end time which is nice. It has quite a complex ui with little guidance making it hard to use and unintuitive.

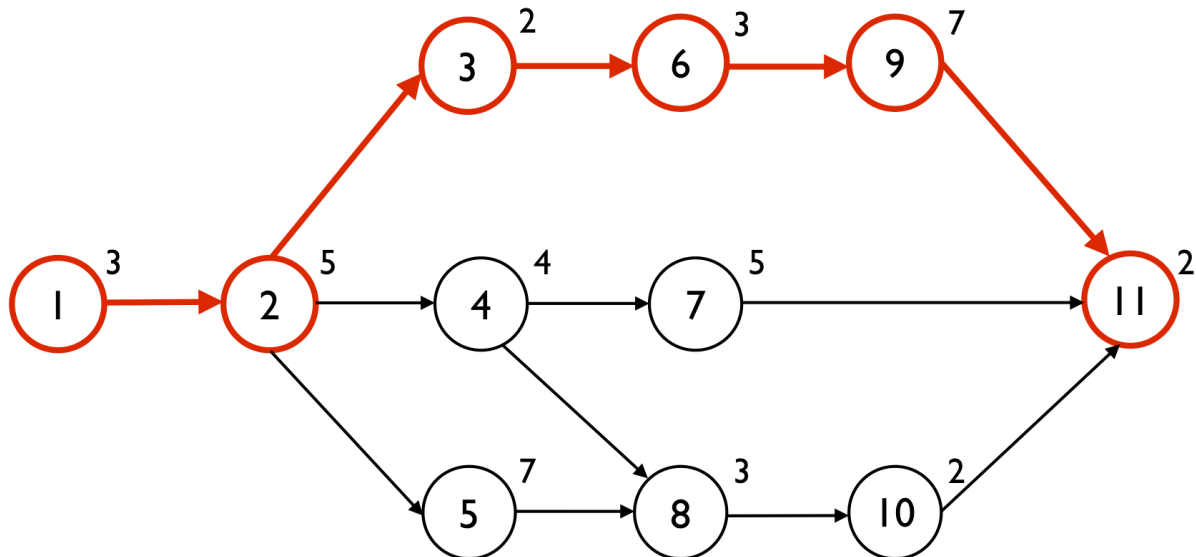
In conclusion the "meal" concept is good with ingredients and timings, as well as the automatic calculation of timings but would be better if the ingredients were relative to each other. A more intuitive ui is needed.

Ways of doing the timings for recipes

The recipes will have to store data for the timings. The data that needs to be stored will be: what each process in the recipe does, the order they happen in, how long they take. There are various methods for doing this: gantt charts/weighted networks, multidimensional arrays or dictionaries.

Using a multidimensional array will work by storing a list of processes, each process will be its own list. Each process will store the information about that process (what it is dependent on, how long it takes, e.t.c.). The use of an array will make it simple to use and expand, however it will mean the need for repeated data or having to calculate the same bit of data each time (e.g. which processes rely on process A or fastest time to get to certain points in the recipe).

Gantt charts and Weighted networks [[007](#) - [008](#)]



Gantt charts may be a way which I can set up the recipes. It is a type of bar chart which contains information about timings of activities and dependencies of activities. It would be an easy to understand way of visualising the data and not difficult to store with the use of weighted networks.

Weighted networks are a collection of 'nodes' and 'paths' each path is an activity, it has a weight (in this case time taken) and information on what needs to be done before it (its dependencies). This could be stored in a database and pulled when needed to the timing app to be displayed as a gantt chart. This would allow for things to take longer than expected /less time and everything adjust accordingly.

Using this would also allow for it to be set up in a way everything finishes as late as possible/same time as so all the food is warm. This can also be used to make it so the recipe finishes by a certain time or starts at a certain time giving a lot of flexibility to the user.

In conclusion I will use gantt charts and weighted networks to store information about timings in the recipes due to their flexibility, intuitively for the user.

Existing tools

There are 2 main tools I need to build this app: one to help build the application and one to manage the database system. The app framework will

App Framework

Django

Django is a web framework for python, being a web framework it would allow for easy implementation to the web and to an app which is useful for delivering the product to a wide range of clients. As it is developed with python (a scripting language designed

for rapid prototyping) and as I have a higher degree of knowledge with python than other languages, it will be easier and faster to develop. I can focus on important features and user experience. It is also open source with good documentation and community support which would allow for more control when developing the app and help where needed.

Kivy

Kivy is an application development framework for python, it can be deployed to windows, linux, macos, ios and android. This is very useful as it can be deployed to an overwhelming majority of operating systems allowing it to reach a wide range of clients. It uses a mix of python and xml, python is a language that I have a higher degree of knowledge with making it easier and faster to develop. The xml will also allow for rapid development and easy customization of the UI. Kivy forces you to write in a very object oriented way due to its class system and xml which will result in well organised and structured code making debugging and making modular code very easy allowing for the development to take less time and the code to be more optimised and faster. Kivy is also open source with a large community for support and good documentation. This will make development easier and allow me to change the way Kivy functions if needed to improve the application.

Android studio

Android studio is an android app IDE, it uses kotlin which is a language im not familiar with making development take longer and giving me less time to focus on making the application well. Kotlin is a much faster language than python though as it is compiled meaning the application will run alot faster and smoother resulting in a better user experience and it is designed for android apps meaning the app should run better in general. It does have easy to design visuals, using a WYSIWYG system which will make it quicker and easier to design an intuitive UI. allowing me also to focus on other features and becoming familiar with the language. It has good documentation and parts of it are open source meaning I have more control over how to make the app and will have support when making it. However it will only compile to android limiting the range of clients I can deliver too.

Database system

MongoDB

MongoDB is a noSQL document oriented database system, it is designed for scalability and flexibility. It sacrifices some ACID properties for flexibility meaning it has less secure transactions. It is often used for local deployment but can support other types such as cloud based deployment, this would be good for the option to scale up in the future to reach a larger number of people. However it is designed to handle large volumes of data which, when scaling up, would be very useful due to the ability for any user to upload recipes to the database. I have no experience with noSQL databases which would mean more time spent learning tools and less designing and building the application. MongoDB is open source with a large community for support which would aid in development.

MySQL

MySQL is a table based database system, it is designed for data integrity, scalability and high performance. It does this by satisfying ACID properties which is important as the database will store important information such as usernames and passwords. If features such as integration with supermarkets are implemented then sensitive data might be stored such as supermarket account information and potentially order history. This data is important and so should not be processed without a transaction satisfying ACID properties, failing to do so could result in: incorrect items in baskets which would result in users purchasing the incorrect items; data getting mixed when two people update their basket at the same time. It also allows for relational databases which makes complex relationships between tables easier. This makes development easier and will allow me to implement more features improving the user experience. However, it can be very resource intensive which isn't ideal for prototyping and would make it a better system to switch to for a final product. A more lightweight system might be more suitable for prototyping.

SQLite

SQLite is a table based database system much like MySQL however it is much more lightweight allowing it to take up less space which is useful for prototyping. MySQL can also be used in a lightweight configuration but SQLite is serverless and embedded, this will mean it is easier to set up for prototyping and testing as well as running faster but does mean it isn't suitable for a final product due to the need for a server based system. It also is easy to set up making it easy to use for small-scale apps or for prototyping. Like MySQL it satisfies ACID properties allowing for reliable transactions. However it isn't designed for large scale systems and so for a final product it wouldn't be appropriate due to the potentially large number of users and calls to the database.

In conclusion for the app framework Kivy is most suitable, using the mix of python and xml will allow for rapid development and will force the code to be well structured and easy to adapt later on. It also has a large community and is open source allowing for more flexibility when developing with the support to do so. For the database: for a prototype SQLite is most suitable, satisfying ACID properties making it a reliable system whilst also being lightweight and easy to setup. Later on this would be switched to MySQL allowing for vertical scalability and higher performance.

Requirements

There will be user and developer requirements for this project and can both be split into the hardware and software requirements.

Hardware

For the users they will require a smartphone, smartphones make up about 50% of the worldwide market share vs tablets and desktops [[004- Statcounter \(03/06/2023\)](#)]. As well as

that, they are smaller and more convenient for use in the kitchen. A large number of stakeholders said they would want it to work on a tablet or mobile device. There must also be a wifi chip to allow the user to connect to the servers to get the recipes.

For development a laptop or computer will be needed for the computational power and ease of use when developing software. An android phone of similar specifications to those that consumers have in order to test the software, is also required. Also when deploying the application, server space will be required to host it. When testing a server won't be required.

Software

Android has a 70.93% market share in phone operating systems [\[005 - Statcounter \(29/04/2023\)\]](#) with 87.07% [\[006 - Statcounter \(29/04/2023\)\]](#) of those being Android 9 and up. I will support android 9 and upwards meaning the app will be accessible to about 60% of phones. The app itself will contain all other software required.

For development any operating system which can run a python ide and file manager will be needed for the programming. A database editor or command line tool with sql will also be needed to build and test the database. The developer will also need a compiler which can build to an apk file for android 9 and up.

Solution

Limitations of a computational solution

Some limitations of doing this using a computational solution are the tractability of the algorithms required. For making recipes, using a weighted graph solution will mean requiring algorithms to generate it then one to perform critical path analysis. This is later used to generate a gantt chart when displaying the recipe. These 2 sets of algorithms need to be tractable as otherwise the user will not be able to create or use recipes in a reasonable time frame. Most shortest path algorithms will have a high time-complexity, Dijkstra's has one of $\Theta(|E|+|V|\log |V|)$. However, most recipes will be represented by small graphs due to the real world feasibility of the recipes and so the algorithms will be feasible, this applies to all of the algorithms that will be required.

The solution is limited by scalability, due to the scope of the project the application cannot be tested on a large scale with a big user base. The application won't be suitable to deploy on a large scale due to this inaccessibility to large scale testing and also due to the lack of infrastructure.

While a finished product would interface with an api to fetch nutritional information, as researched in stakeholder interviews, it goes beyond the scope of the project and so will not be implemented. This again limits the solution as testing cannot take into account all the features the finished product would have, reducing the opportunities for improvement on the solution in this area.

The app won't be able to support recipes which take over 24 hours in length, there may be issues with the phone turning off the switching tab and it losing progress over a large period of time and is hard to account for everything that could happen in such a large time frame. It is also unrealistic for most recipes which take that long to need a timer of this complexity as the cook would not be doing things for all the time when it is happening. e.g. slow cooking meat

Essential Features

The app should contain some essential features to function correctly and act as a good solution.

First the app should have an account system which allows the user to create and log into accounts. These accounts should also link to things the user does on the app such as making and liking recipes.

Second the app should have a recipe making system, this should be intuitive to use and/or have descriptions on how to use it. The user should be able to add their recipe to the database, these recipes should be shown on the recipe browser and their profile.

Finally the app should have a timer interface for each recipe which is made by the user when making the recipe. The timer should be easy to use and displayed as a dynamic gantt chart.

Success Criteria

Criteria	How to evidence
The user should be able to log into their account	Screenshot
The user should be able to create an account	Screenshot
The usernames can only be used once and should not be null	Screenshot
Passwords should not be null	Screenshot
Passwords are hashed	Screenshot and link to code
After logging in the user should be able to see a list of recipes from the database	Screenshot
Each recipe should be shown in a recipe	Screenshot

card	
Each recipe card will show the name of the recipe, any related info and whether or not the user has liked that recipe.	Screenshot
When a 'recipe card' is clicked on it should take the user to the recipe page	Screenshot
The like button should be able to be toggled and save its state to the server	Screenshot of interface and database
The profile view button should take the user to the profile of the user who made the recipe when clicked. This page should also be accessible through a 'My profile' tab.	Screenshot
The profile view should show liked recipes if the profile is of the user currently logged in	Screenshot
The nutrition button on the recipe view interface should give nutritional information in the recipe.	Screenshot
The recipe making interface should have 3 tabs for: general details, recipe processes and then uploading to database	Screenshot
The recipe making interface should allow the user to input a name, basic description in the general details section	Screenshot
The processes interface should let the user input data about timing, dependencies and methods.	Screenshot
There should be a place to add ingredients to the recipe which links to a database of ingredients.	Screenshot
The final screen of the recipe making interface should allow the user to check if the recipe is valid and then save it to the database.	Screenshot
There should be a button which lets the user open the timer interface for the recipe.	Screenshot

The app should let the user set a start/end time for the recipe.	Screenshot
The app should generate a gantt chart timer from the network and nodes data	Screenshot
The timer interface should show upcoming and ongoing processes in labelled boxes	Screenshot
The timer interface should adapt to how long different processes take	Screenshot

Design

The application is split up into 3 separate components which will be designed separately but will work together. Due to the nature of the Kivy system the GUI and code will be developed very closely, this is due to the XML heavily used in Kivy. This is talked more about in the [code design](#). The database can be completely developed separately, this is necessary as it will allow for easier development as one section of the application being developed does not rely on another section having already been developed. It will also make debugging/fixing or updating different sections easier as they are kept separate.

Database Design

V1

This is the initial design for that database, each process had a separate record in the database, however there was no link to the recipe and so it would not work. As well as that, many ingredients could be used for one process. I tried to account for this by adding TBL_RecipeIngredients, however this was done incorrectly as still only one ingredient could be used per recipe and so didn't resolve the issue.

tbl_Users					
INT userID	TEXT username	TEXT password			
a non-null unique id value	a user entered username stored as text	a user entered password, hashed then stored as text			
tbl_UserRecipes					
INT userRecipeID	INT userID	INT recipeID	BOOL owner		
a non-null unique id value	foreign key	foreign key	a boolean, true if user (userID) is owner of the recipe (recipeID)		
tbl_Recipes					
INT recipeID	TEXT name	TEXT description	TEXT method	INT aproxTime	
a non-null unique id value	foreign key	brief user entered description of the recipe	user entered written method on how to make the recipe	user entered approximate time the recipe will take in minuets (converted by client to appropriate)	
tbl_RecipeIngredients					
INT recipeIngredientID	INT ingredientID	TEXT quantity			
a non-null unique id value	foreign key	user entered text stating the ammount of something needed (with units if appropriate)			
tbl_Processes					
INT processID	INT recipeIngredientID	INT startTime	INT endTime	TEXT description	TEXT timing
a non-null unique id value	foreign key	user entered time in minuets from end of last thing / start of cooking when it needs to start	user entered time in minuets from startTime for this process to take	user entered description for how to do this process	2, 2-digit hexadecimal numbers in the format AA-AA, the first number giving a channel number and the second giving a order number
tbl_Ingredients					
https://esha.com/products/nutrition-database-api/					

V2

This is the second version of the database design, some issues like having only one ingredient per process have now been resolved. This was done by fixing TBL_RecipeIngredients and adding TBL_ProccessIngredients, these tables allow for any number of ingredients to be associated with a process or recipe. I have also tried to add conditions to the processes so it could be saved as a network, however it is a complex system and due to the time limitations of this project something more simple is needed.

tbl_Users				
INT userID	TEXT username	TEXT password	TEXT description	
a non-null unique id value	a user entered username stored as text	a user entered password, hashed then stored as text	user description shown on profile	
tbl_UserRecipes				
INT userRecipeID	INT userID	INT recipeID	BOOL owner	
a non-null unique id value	foreign key	foreign key	a boolean, true if user (userID) is owner of the recipe (recipeID)	
tbl_Recipes				
INT recipeID	TEXT name	TEXT description	INT aproxTime	TEXT imgType
a non-null unique id value	foreign key	brief user entered description of the recipe	user entered approximate time the recipe will take in minuets (converted by client to appropriate)	image file type (e.g. 'jpg') if None then defaults to defaultRecipeImage.jpg otherwise it is userID-recipeID.imgType
tbl_RecipeIngredients				
INT recipeIngredientID	INT recipeID	INT ingredientID	INT quantity	TEXT units
a non-null unique id value	foreign key	foreign key	ammount of ingredient	unit
tbl_Processes				
INT processID	BOOL conditions	INT timeTaken	TEXT description	
a non-null unique id value	are there any other process that need to be done before this?	how long in minuets does it take	user entered description for how to do this process	
tbl_ProcessIngredients				
INT processIngredientID	INT processID	INT recipeIngredientID		
a non-null unique id value	foreign key	foreign key		
tbl_ProcessConditions				
INT processConditionID	INT processIDCondition	INT processIDCondition		
a non-null unique id value	foreign key (processID)	foreign key (processID)		
tbl_Ingredients				
INT ingredientID	https://esha.com/products/nutrition-database-api/			

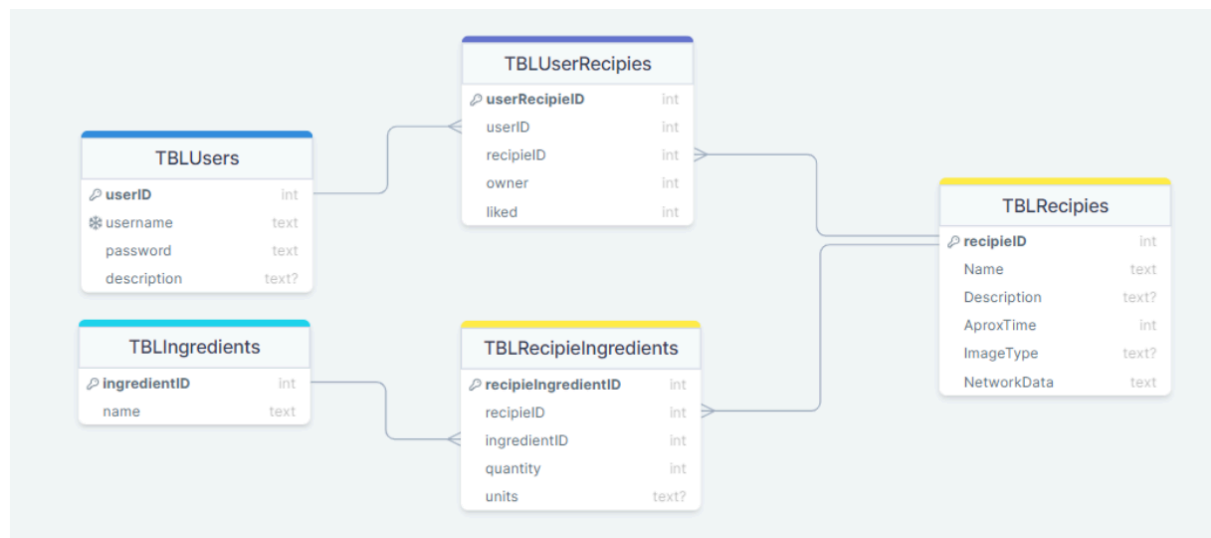
V3

This is the final design for the database, it can store all the information needed in the network datatype and is simple to use allowing for rapid development which is necessary due to the time limits of the project.

This system won't allow for specific ingredients to be associated with the processes. If the project were to continue, a more complex system like that of v2 would be adopted to allow for more functionality.

tbl_Users					
INT userID	TEXT username	TEXT password	TEXT description		
a non-null unique id value	a user entered username stored as text	a user entered password, hashed then stored as text	user description shown on profile		
tbl_UserRecipes					
INT userRecipeID	INT userID	INT recipeID	BOOL owner		
a non-null unique id value	foreign key	foreign key	a boolean, true if user (userID) is owner of the recipe (recipeID)		
tbl_Recipes					
INT recipeID	TEXT name	TEXT description	INT aproxTime	TEXT imgType	JSON
a non-null unique id value	foreign key	brief user entered description of the recipe	user entered approximate time the recipe will take in minutes (converted by client to appropriate)	image file type (e.g. 'jpg') if None then defaults to defaultRecipeImage.jpg otherwise it is userID-recipeID.imgType	network data
tbl_RecipeIngredients					
INT recipeIngredientID	INT recipeID	INT ingredientID	INT quantity	TEXT units	
a non-null unique id value	foreign key	foreign key	amount of ingredient	unit	
tbl_Ingredients					
INT ingredientID	https://esha.com/products/nutrition-database-api/				

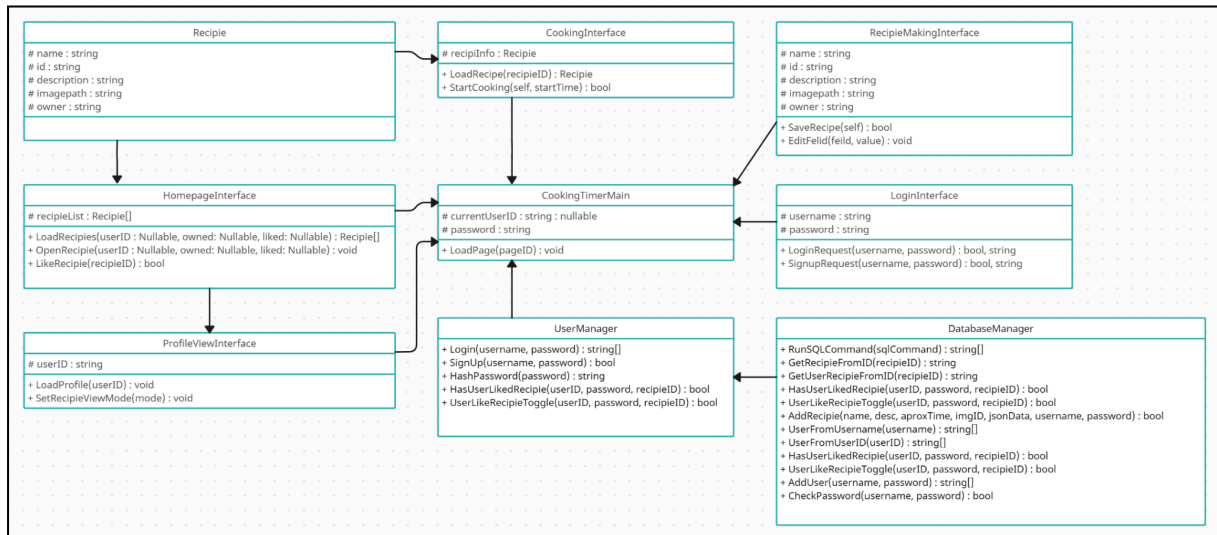
Relationship entity diagram



This shows the final design for the database as a relationship entity diagram. The main tables are TBLUsers and TBLRecipes, TBLUsers stores user information like username and the hashed password. TBLRecipes stores recipe information like name, description and the timing information. TBLUserRecipes will make connections between users and recipes, it stores who is the owner and who has liked the recipe. The owner of the recipe is the one who made it by default and cannot be changed. If the project were to continue functionality for editing the recipe would be added but that is outside the scope of this project.

Code design

Class Diagram



The code will be object oriented and take a singleton approach, this is due to 2 reasons; one being it will allow for easy management of data and instances of parts of the application, for example the login information will be stored in the singleton and so will never be deleted whilst the application is running which is important as the user wouldn't want to login every time they change screen, whilst also allowing for all the data for a recipe to be stored with the instance of the timer class and then when that instance is no longer being used, the garbage collector will remove all the data together. It's a nice way of organising and managing the code, also however, as mentioned in the [Kivy](#) research section, the way Kivy is designed very much encourages the use of object oriented and singleton approaches, it does this with its use of XML. It uses objects in XML to represent different elements on the screen, these objects can be linked to classes in the code, this heavily promotes an object oriented approach as otherwise scripting those elements becomes very difficult.

The main class, `CookingTimerMain`, is a singleton meaning it manages all other aspects of the application, including users and the GUI. `currentUserID` is nullable to allow for the user to login without an account and uses a string due to the possibility for use of letters, symbols and numbers. The password is also a string for the same reasons, this is required as limiting the types or symbols or length of the password would result in less secure passwords. The `LoadPage()` function loads different interfaces for a given `pageID`, such as recipe viewer, home page or profile viewer.

The `HomepageInterface` class is one of the main interfaces for the application, it contains the recipe browser and viewer. The `recipeList` is a list of `Recipe` instances, this is generated in the initialisation using the `LoadRecipes()` function and is displayed on the browser as a 'banner'. It is stored in a list of class instances as the list makes it able to manage any number of instances allowing for more modular code, the use of class instances makes it easy to manipulate data for each recipe separately and easier to read. From this browser the recipe can be opened, liked, and a few key bits of information are shown. An image of the recipe will be displayed

if submitted to the database, there will also be the name and the first part of the description displayed.



Design for a recipe 'banner' as shown in the [Interface Design](#)

The LikeRecipe() function can toggle the like button on the recipe, this function calls the UserLikeRecipeToggle() function in the DatabaseManager. When the 'banner' is clicked the OpenRecipe() function will open the recipe in a recipe viewing interface, from here, the rest of the recipes information will be shown as well as the ability to view the creators profile page.

The ProfileViewInterface class manages viewing a user's profile, this profile includes information about which recipes they have liked and also which they have made. The UserID of the user being viewed is stored in a string, as after it has been generated it is easier to use as a string and as it should not be affected by mathematical operations, it prevents that from happening. LoadProfile() will fetch all the information from the database. The SetRecipeViewMode() function switches between viewing their liked recipes and the recipes they have made.

The CookingInterface class will manage the cooking timer aspect of the application, it will lead the recipe from the database into recipeInfo using LoadRecipe() and display it as a gantt chart. It will remain idle until the user presses a start button causing the StartCooking() function to run. This will manage the timer when it is running and make sure everything is adjusted when needed in order for the timer to function correctly.

The RecipeMakingInterface class will manage making a recipe. name, id, description, imagePath and owner will be stored as strings. All but id and owner can contain numbers, symbols and characters and so a string is needed. The value of these variables will be determined by the values inputted by the user to the corresponding fields. id and owner are also stored as strings for the same reason UserID is stored as a string in ProfileViewInterface, it should not be changed via mathematical operations and so a string is most appropriate. The EditField() will change data in the recipe when it is being made by the user. Owner stores the user id of the user making the recipe. SaveRecipe() will send a request to the DatabaseManager via UserManager which will save the recipe to the database.

LoginInterface will manage logging into the account and creating accounts. username and password will store the values inputted by the user to corresponding fields and will be sent to the database as a signup or login request with the

corresponding functions. SignupRequest() will also check if the username already exists/ password is valid and output those errors to the user so they can fix it. LoginRequest() will only output "login unsuccessful" as giving details to which part of the login is incorrect would be a security risk as a user with malicious intent could use this when guessing usernames and passwords to aid them. Also a password can be used multiple times by different users and so outputting 'username incorrect' would not only be a massive security risk but also be difficult to implement.

UserManager is a helper class to DatabaseManager, UserManager will run on the client and DatabaseManager will run on the server. In the project they will both run on the same machine as having a server is outside the scope of the project. UserManager will send requests to DatabaseManager to run sql and fetch data from or edit the Database. Depending on the request DatabaseManager might require a password and userID, such as if the user wants to add a recipe to their profile.

Network Generation Algorithm

CalculateDependencyTrees and TraverseNodeCircularDependency algorithms

When the user makes a recipe they will input the data into an interface (shown in the [interface design](#) section) which will then run that through the following algorithm. This creates dependency trees, a data type composed of lists which defines which elements of the recipe rely on others to be finished. This will allow it to be processed in the [TraverseProcessesBuildNetwork](#) algorithm creating a network which can have other algorithms applied to it. I have talked more about why this is necessary in the [TraverseProcessesBuildNetwork](#) section.

```
function CalculateDependencyTrees(processes) {
    #make sure there at least one starting procces
    startingProcesses = []
    for i in processes {
        if i.get("conditions") == "" {
            startingProcesses.append(i.get("id"))
        }
    }

    if len(startingProcesses) < 1 {
        return -1, "Invalid recipie, You need at least one starting proccess"
    }

    #get list of dependencys
    dependencys = {}
    for i in processes {
        tmp = i.get("conditions").split('\n')
        dependencys[i.get("id")] = list(filter(lambda a: a != '', tmp))#remove '' values
    }

    startingNodes = []

    for i in dependencys {
```

```

        if dependencys[i] == [] {
            startingNodes.append(i)
        }
    }

    #get dependencys backwards
    backDependencys = {}#{"start" : startingNodes}
    for i in dependencys.keys() {
        tmp = []
        for j in dependencys.keys() {
            if str(i) in dependencys[j] {
                tmp.append(j)
            }
        }

        backDependencys[i] = tmp
    }

    allNodesVisited = []

    #make sure there is no circluar dependencys
    for i in startingNodes {
        tmp, nodesVisited = TraverseNodeCircularDependencys(i, backDependencys, [], [])

        if tmp == False {
            #Invalid recipie, You need at least one starting proccess
            return -1
        }

        allNodesVisited += nodesVisited
    }

    allNodesVisited = list(set(allNodesVisited))
    allNodesVisited.sort()

    tmp = list(dependencys.keys())
    tmp.sort()

    if allNodesVisited != tmp {
        #Invalid recipie, There are circular dependencys
        return -1
    }

    endingNodes = []

    for i in dependencys {
        if backDependencys[i] == [] {
            endingNodes.append(i)
        }
    }

    dependencys['start'] = []
    backDependencys['start'] = []

    for i in startingNodes {
        dependencys[i].append('start')
        backDependencys['start'].append(i)
    }

    dependencys['end'] = []
    backDependencys['end'] = []

    for i in endingNodes {
        dependencys['end'].append(i)
        backDependencys[i].append('end')
    }

    return [dependencys, backDependencys, startingNodes, endingNodes]

```

```
}

function TraverseNodeCircularDependency(currentPoint, tree, pointsToAvoid, pointsVisited)
{
    pointsVisited.append(currentPoint)
    if tree[currentPoint] == [] {
        return True, pointsVisited
    }

    if currentPoint in pointsToAvoid {
        return False, None
    }

    pointsToAvoid.append(currentPoint)

    for i in tree[currentPoint] {
        tmp, foo = TraverseNodeCircularDependency(i, tree, pointsToAvoid[:],
pointsVisited)
        if tmp == False {
            return False, None
        }
    }

    return True, pointsVisited
}
```

TraverseProcessesBuildNetwork Algorithm

This algorithm takes in the dependency tree, this is computed by the program from the input from the user when making a recipe, and then generates a network. This makes computing data based on the recipe a lot easier as many of the algorithms needed have already been designed around networks such as dijkstra's shortest path algorithm. This is necessary as it will reduce the amount of algorithms that need to be designed making the project feasible in the given time frame and also will allow for me to use the most efficient algorithms developed reducing the time complexity of my code. It has a time complexity of $O(n^4)$ limiting the size of the recipe, however due to the real world limitations to the size of recipes, such as time it takes to cook, they won't be of a size where they are intractable for the algorithm. Other algorithms which rely on the network such as dijkstra's algorithm are of lower time complexity, dijkstra's is of $O(n^2)$

It works by iterating through each process in the recipe checking if its dependencies have been added and if so then adds it to the network. When adding it to the network it checks for things like if its dependencies go to the same node, if it shares any dependencies with other processes and if dummies need to be created. It then does the appropriate actions and adds/deletes edges and nodes as needed.

Computer Science Project - Recipe Manager & Timing Application [top](#)

```
function TraverseProcessesBuildNetwork(startingNodes, endingNodes, dependencys,
backDependencys) {
    dictionary network = {}          # dictionary {"processID"/"dummyID": [startPoint,
endPoint]}
    dictionary nodes = {}           # dictionary {"ID": ["conditionIn", "conditionIn",
...]}
    #conditions are the process that must be completed for it to start, ignore dummies just
say which process

    int currentNode = 1             # this is used to keep track of which proccessID s have been
used
    int currentDummy = 0            # this is used to keep track of which dummyID s have been used

    #nodes - points in the network
    #edges - processes or dummies in the network

    bool proccessing = True        # used to keep the loop running until the network has been
generated

    int cycle = -1                 # keeps track of cycle number
    str lastCycle = ""             # keeps track of last cycles networkdata (to check if network
has finished building)

    while proccessing {
        cycle = cycle + 1
        for i in dependencys {
            //ignore if already added
            if i is in network {
                continue to next iteration
            }

            #can this edge be added yet (has its dependencys been added)
            vailidEdge = True
            for j in dependencys[i] {
                if j is not in network {
                    vailidEdge = False
                    break out of loop
                }
            }

            #add edge
            if vailidEdge == False {
                continue to next iteration
            }

            #are its dependencys going to same node?
            toSameNode = True
            firstDepend = '-1'

            if len(dependencys[i]) > 0 {
                firstDepend = dependencys[i][0]
            }

            for j in dependencys[i] {
```



```

        if network.get(j)[1] != network.get(firstDepend)[1] {
            toSameNode = False
        }
    }

#Y - add edge from that node
if tmp {
    foo = network.get(firstDepend)
    if foo != None {
        startNode = foo[1]
    } else {
        startNode = "0"
    }
    network[i] = [str(startNode), str(currentNode)]

    if nodes.get(currentNode) == None {
        nodes[str(currentNode)] = [i]
    } else {
        nodes[currentNode].append(i)
    }
    currentNode += 1
}

#N - continue
else {
    #does one/more of its dependencys act as a dependency for something else?
    sharedDependencys = []

    for currentProccessDependency in dependencys[i] {
        if len(backDependencys[currentProccessDependency]) > 1 {
            sharedDependencys.append(currentProccessDependency)
        }
    }

    if len(sharedDependencys) != 0 {
        #Y
        #make listA (list of something elses that share the same set of
dependencies)
        listA = []
        #make listB (list of something elses that DON'T share the same set of
dependencies)
        listB = []

        #get list of something elses somethingElseID
        somethingElses = []
        for j in sharedDependencys {
            tmp = []
            for k in backDependencys[j] {
                if k not in somethingElses and k != i {
                    somethingElses.append(k)
                }
            }
        }

        #for each something else:

```

```
for j in somethingElses {
    #does that something else share the same set of dependencies?
    if sorted(dependencys[j]) == sorted(dependencys[i]) {
        #Y - Add to listA
        listA.append(j)
    } else {
        #N - Add to listB
        listB.append(j)
    }
}

#create temporary listB, make empty (for dummy nodes)
tmpDummy = {}
#{ "dummyID" : startNode}

#for each item in listB:
for j in listB {
    #add a dummy to tmpDummy
    nodeOrigin = network.get(j)

    if nodeOrigin == None {
        continue to next iteration
    }

    for k in backDependencys {
        if j in backDependencys[k] {
            nodeOrigin = network[k][1]
        }
    }

    tmpDummy["dummy" + str(currentDummy)] = nodeOrigin
    currentDummy = currentDummy + 1
}

#combine dependencys in temporary node to real node
nodeID = ''

for j in dependencys[i] {
    if j not in sharedDependencys {
        nodeID = network.get(j)[1]
        break out of loop
    }
}

if nodeID == '' {
    nodeID = currentNode
    currentNode = currentNode + 1
}

nodes[nodeID] = list(set(dependencys[i]))
#add dummies from dependencys to real node (tmpDummy)
```

```

        for j in tmpDummy {
            network[j] = [tmpDummy[j], str(nodeID)]
            nodes[nodeID].append(j)
        }
        #combine other dependencys (remove old nodes)
        for j in dependencys[i] {
            if j not in sharedDependencys and (network[j][1] != nodeID) {
                nodes.pop(str(network[j][1]))
                network[j][1] = nodeID
            }
        }
        #add the new edge and node
        network[i] = [str(nodeID), str(currentNode)]
        nodes[str(currentNode)] = [i]
        currentNode += 1
    }

else {
    #N - combine dependencys to same node then add edge to node (update node
info)
    nodesToCombine = []

    for j in dependencys[i] {
        nodesToCombine.append(networkdata[j][i])
    }

    newNodeInfo = []
    for j in nodesToCombine {
        for k in nodes[j] {
            newNodeInfo.append(k)
        }
    }

    for j in nodesToCombine[1:] {
        nodes.pop(j)
    }

    nodes[nodesToCombine[0]] = newNodeInfo

    for j in newNodeInfo {
        network[j][1] = str(nodesToCombine[0])
    }

    network[i] = [str(nodesToCombine[0]), str(currentNode)]
    if nodes.get(currentNode) == None {
        nodes[str(currentNode)] = [i]
    } else {
        nodes[currentNode].append(i)
    }
    currentNode += 1
}
}
}

#if all dependencys in network
#Y - proccessing = False

```

```

thisCycle = str([network, nodes])
if thisCycle == lastCycle {
    proccessing = False
}

lastCycle = str([network, nodes])

return network, nodes
}

function TPBN_WhichNodeHasIn(nodes, id) {
    for i in nodes {
        if id in nodes[i] {
            return i
        }
    }
    return -1
}

function TPBN_WhichNodeHasOut(network, id) {
    for i in network {
        if id == network[i][0] {
            return i
        }
    }
    return -1
}

```

Test data

Algorithm	Type	Notes	Test Data	Expected output
SubmitLoginDetails	Normal	User 'James' exists with password '1234' This will show it works for any working login attempt	['James', '1234']	Successful login
	Normal	User 'James' exists with password '1234' This will show it works for login attempts with the wrong password	['James', 'abcd']	Failed login

	Normal	User 'James' exists with password '1234' User 'Garry' does not exist This will show it works login attempts where the user doesn't exist	['Garry', '1234']	Failed login
SubmitSignUpDetails	Normal	User 'James' does not exist This will show it works for any working login attempt	['James', '1234', '1234']	Successful signup
	Normal	User 'James' exists	['James', '1234', '1234']	"Username already exists"
	Erroneous		['', '1234', '1234']	"Invalid username"
	Erroneous		['James', '', '']	"Invalid password"
	Erroneous		['James', '1234', 'abcd']	"Passwords don't match"
CalculateDependencyTrees	Normal	Data validation will prevent invalid syntax being input	[{'id': '0', 'activePassive': '0', 'approxTime': '5', 'conditions': '', 'description': ''}]	[{'0': ['start'], '1': ['0'], 'start': [], 'end': ['1']}, {'0': ['1'], '1': ['end'], 'start': ['0'], 'end': []}, ['0'], ['1']]
	Normal	Checks if works for dummy nodes	[{'id': '0', 'activePassive': '0', 'approxTime': '5', 'conditions': '', 'description': ''}, {'id': '1', 'activePassive': '0', 'approxTime': '5', 'conditions': '', 'description': ''}, {'id': '2', 'activePassive': '0', 'approxTime': '5', 'conditions': '0\n1', 'description': ''}, {'id': '3', 'activePassive': '0', 'approxTime': '5', 'conditions': '1', 'description': ''}]	[{'0': ['start'], '1': ['start'], '2': ['0', '1'], '3': ['1'], 'start': [], 'end': ['2', '3']}, {'0': ['2'], '1': ['2', '3'], '2': ['end'], '3': ['end'], 'start': ['0', '1'], 'end': []}, ['0', '1'], ['2', '3']]
	Erroneous	Won't work for recipes with circular dependencies	[{'id': '0', 'activePassive': '0', 'approxTime': '2', 'conditions': '1', 'description': ''}, {'id': '1', 'activePassive': '0', 'approxTime': '2', 'conditions': '0', 'description': ''}]	-1 / None
TraverseProcessesBuildNetwork	Normal		[{'0': ['start'], '1': ['0'], 'start': [], 'end': ['1']}, {'0': ['1'], '1': ['end'], 'start': ['0'], 'end': []}, ['0'], ['1']]	{ "network": { "start": ["0", "1"], "0": ["1", "2"], "end": ["2", "3"] }, "nodes": { "1": ["start", "2": ["0"], "3": ["end"]], "criticalPath": ["start", "0", "end"], "earlyLateTimes": { "0": [0, 0], "3": [5, 5], "1": [0, 0], "2": [5, 5] }, "activeProcesses": [] }

	Normal	Checks if works for dummy nodes	<pre>{'0': ['start'], '1': ['start'], '2': ['0', '1'], '3': ['1'], 'start': [], 'end': ['2', '3']}, {'0': ['2'], '1': ['2', '3'], '2': ['end'], '3': ['end'], 'start': ['0', '1'], 'end': []}, ['0', '1'], ['2', '3']}</pre>	<pre>{"network": {"start": ["0", "1"], "0": ["1", "2"], "1": ["1", "3"], "dummy0": ["3", "2"], "2": ["2", "4"], "3": ["3", "4"], "end": ["4", "6"]}, "nodes": {"1": ["start"], "2": ["1", "0", "dummy0"], "3": ["1"], "4": ["2", "3"], "6": ["end"]}, "criticalPath": ["start", "1", "2", "end"], "earlyLateTimes": {"0": [0, 0], "6": [10, 10], "1": [0, 0], "2": [5, 5], "4": [10, 10], "3": [5, 5]}, "activeProcesses": []}</pre>
TimerInitialTimingsSetup	Normal	IN - [minRecipeTime, startEndTimeToggle, timeInput] OUT - [startTime, endTime]	[5, 0, '0:0']	['00:00', '00:05']
	Normal	Test for times over 1 hour	[65, 0, '0:0']	['00:00', '01:05']
	Erroneous	Test for times over 1 day	[1445, 0, '0:0']	Throws error, recipe time should not be saved if over one day (discussed in limitations of computational solution)

Data Validation

During signup, the input of usernames and password are validated by the client and then the server. It checks if the username and password meet the specified criteria:

Username	Not null	No bigger than 32 chars
Password	Not null	Passwords match

The same criteria must be met for the during login, however they are not checked as no account can exist that does not match these criteria (assuming the account is created using the application). A username and password is input and only data verification is applied.

In a final product there would be validation to protect against sql injections, however building a comprehensive system to protect against this is outside the scope of this project.

During the recipe creation process there will be data validation which makes sure the recipe being input has the correct data types for each field. Things such as a name and description are required.

The system will also check the recipe is possible for the algorithm to solve, it will filter out recipes with things like circular dependencies, lack of starting/ending nodes. This means only recipes which will work with the network building algorithm will be validated. This is needed as it could crash the program, create impossible to use recipes or get stuck in infinite loops otherwise.

Data Verification

When logging in or signing up, after the login details have been validated, the username and password are verified. If the user is attempting to login, the password is checked to see if it matches the one stored with the username in the database. If the user is attempting to sign up, the username is checked to see if it already exists. If these checks are passed the user logs in or their account is created.

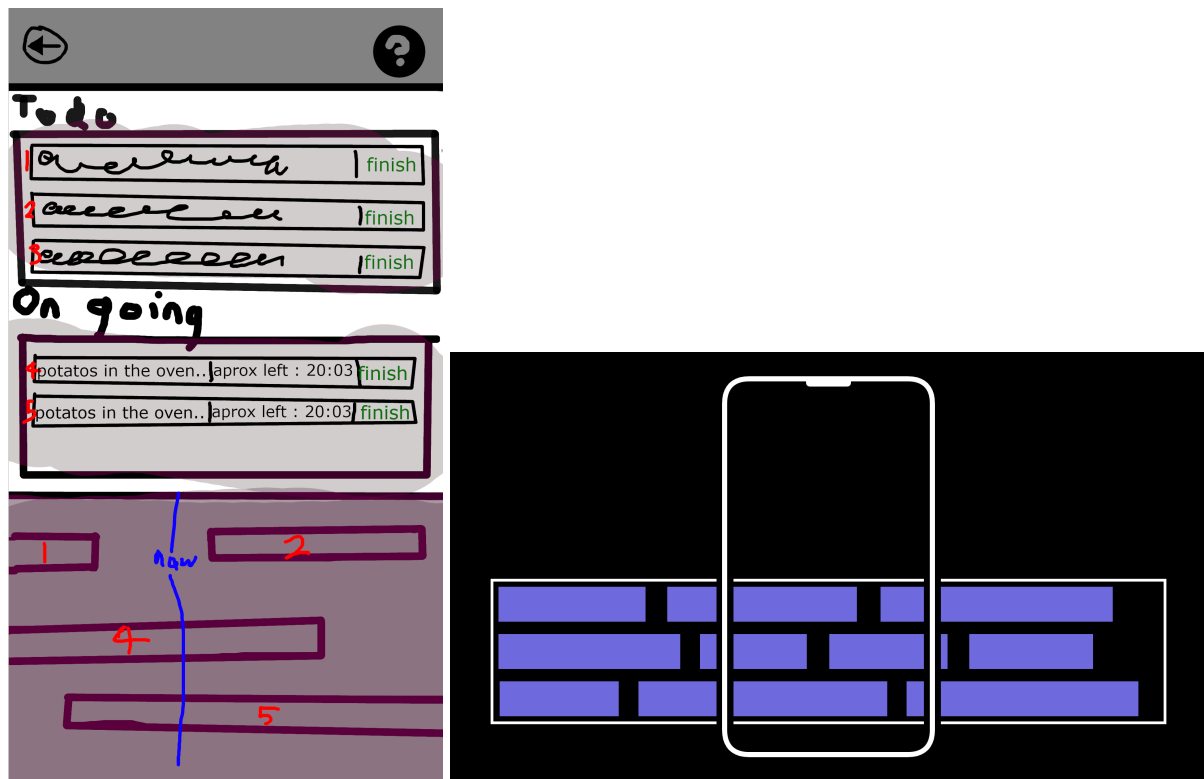
When a client makes a request for a certain recipe or user's information from an ID, the id is verified by the server to make sure the user or recipe exists and in some circumstances it checks the username and password of the client. This is to prevent any user gaining functional access to another user's account, things such as liking or making recipes under another user's username, without the password, will be prevented by this.

Key data structures

Name	Datatype	Notes
Username	string	As mentioned in Class Diagram section of design, username will be a string as any combination of characters (symbols or letters) may be stored in the username
Password	string	Same reasons as the username (see above)
Dependency tree	dictionary	This stores a process Id as a key against a list of processIDs which that

		process depends on. This is used when generating the network and nodes for the recipe network. It is stored like this as it is easy to locate data making the code faster and easier to write. It will allow me to store an unknown quantity of data which is necessary as recipes can have different numbers of processes.
Network	dictionary	This stores a processID as a key against an array, storing a starting and ending node. The nodes are stored as nodeIDs which link to ids in the node data type. I used a dictionary as it makes searching for a certain entry a lot easier, and it will allow for storing an unknown quantity of data which is necessary as recipes can have different numbers of processes. The use of a dictionary also allows for keys in one dictionary to act as values in another dictionary (Nodes) which will make linking the 2 data structures very easy.
Nodes	dictionary	This stores an array with, this stores processIDs of processes which have it as an ending node. I used a dictionary here for the same reasons as the Network as both data structures work together and are very similar. (see above)

Interface Design



The above images show initial designs for how the timer screen should look and function. In the top section there will be lists of ongoing and upcoming stages of the recipe. There will be numbers next to each stage acting like a key, linking those tasks to their block on the timer Gantt chart. This will abstract the visual clutter from the descriptions of the recipes away from the timer section of the screen whilst not removing any functionality. This will also make it easier to see what is upcoming and what you have started. Some stages will be close to the 'now line' as they may not have started yet, but it would be hard to differentiate between these stages and one which has just started, the same thing would happen with stages which have just finished and ones which are waiting to be completed.

Each stage has information on what it is, approximate time until it starts/until it ends and an option to move it onto the next stage (to do -> on going -> finished).

The buttons will be made large and there will be some spacing between the stages in order to make them easy to use. The back and help buttons will also be big and easy to use with symbols which make them easy to understand. The help button will explain how the timer works. These features will make the timer easier to understand and more accessible.



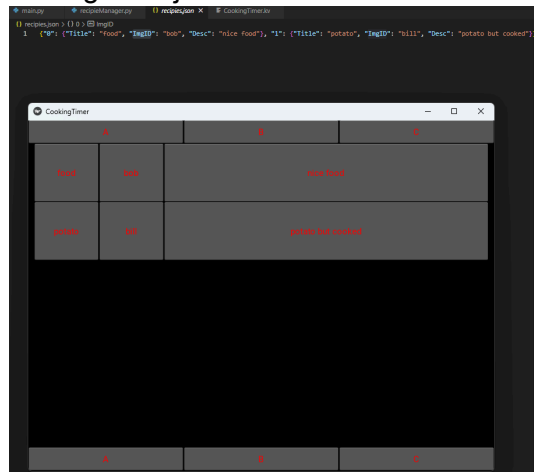
Above is shown one of the final designs for a 'banner' on the home screen, these will be shown, vertically stacked, on the home screen. There will be the name and description shown as well as an image if provided, for this project images will not be fully supported due to the lack of a server, uploading and storing them would be impossible and alternatives would be a good proof of concept. Some recipes will have images manually added to show its functionality however. There will also be a like button which can be toggled on/off to add/remove the recipe to/from the users liked recipes and also the icon will change to indicate whether or not the recipe is currently liked by the user.

The usability of this feature will be good as the icons will be big and stand out, the heart symbol is one of the universal symbols for liking/saving a post online. The thumbs up is also often used but it doesn't indicate saving the post as much as a heart, and a star indicates saving but not liking. A heart indicates clearly saving and publicly liking a recipe. The use of the toggling icon also helps make it clear on whether or not the recipe is liked and if the like was successful.

Development and testing

Image	Notes
-------	-------

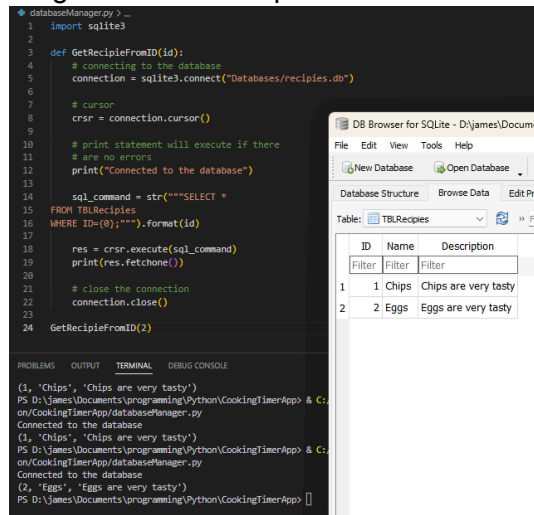
Start of recipe browser, basic recipe loading from json



I built a basic interface in kivy. The CookingTimerMain, HomePageInterface and UserManager were started.

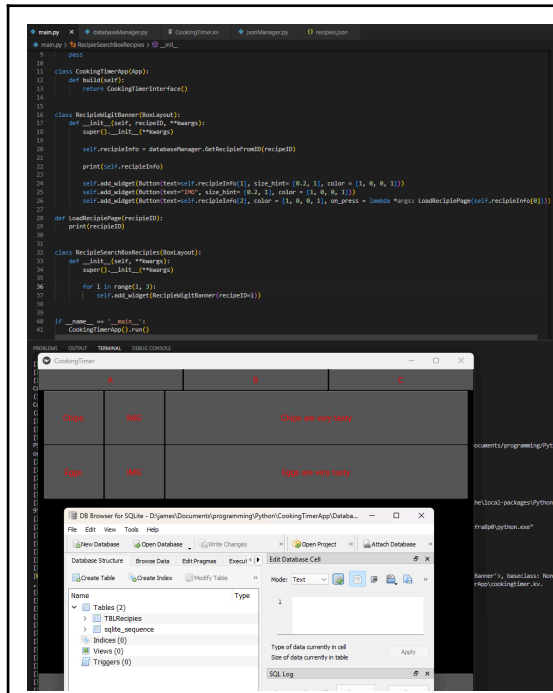
The CookingTimerMain initialises HomePageInterface which calls UserManager to load and return a list of recipes from a temporary json file. I did this because this was quick and easy to get setup, it will allow for a basic frame to build on and test features until the sql database system is implemented.

Set up basic database for testing and integrated it with recipe browser



The json file has been replaced with a sqlite database. It currently only contains the TBL_Recipes as shown in the [database design](#). DatabaseManager has been set up and it manages interactions with the database. The use of the databaseManager is outlined in the [class diagram](#).

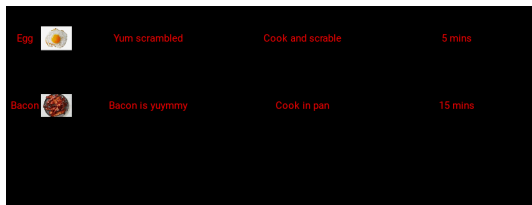
I set up these core features early to allow for everything to be built around them, if they were built later on it could mean I have to redesign a lot of the system around how they are implemented.



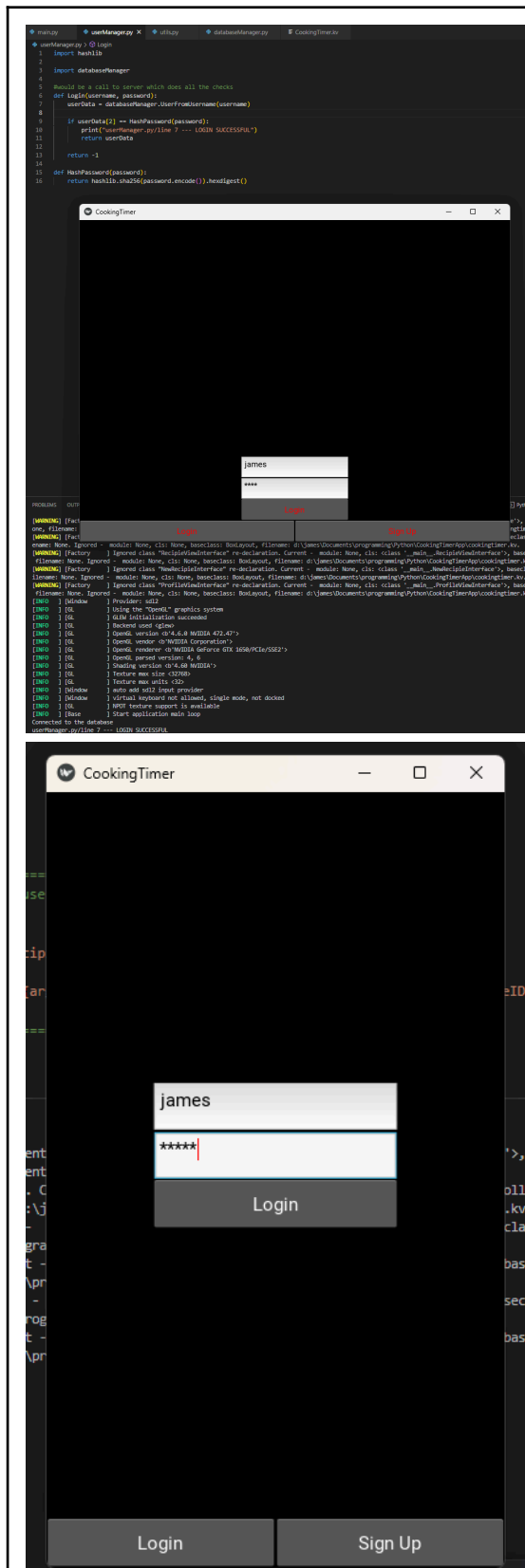
Started work on new design for recipe banners which are shown in recipe browser, they don't currently work



Banners working but still visually need improvement, image loading working too now



The user interface is hard to understand, the text does not stand out and the banners are made up of 3 buttons making it confusing for the user deciding which button to press. I solved this by updating the UI to make it more intuitive. I included all the elements in one button to make it easier to use, made the background black so the text stood out and added images to the banner. The UI will constantly be improved throughout the project as new features are added, if the UI is built to be complete now it would still need to be redesigned later on as new features are added. The need for an intuitive and clear UI is highlighted in the [Interface Design](#) section.



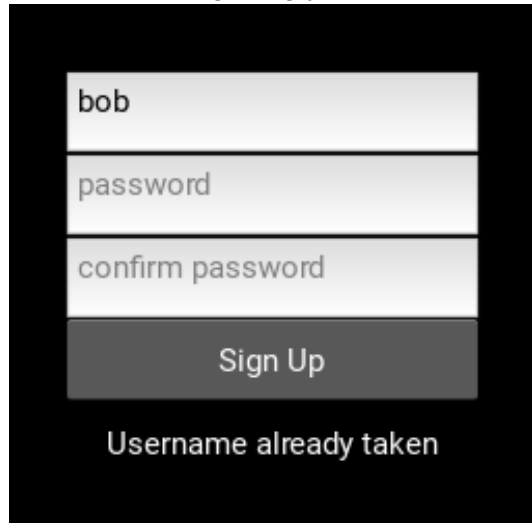
Working login feature, banners for recipes nearly working

Sign up feature added (checks username is unique and password is

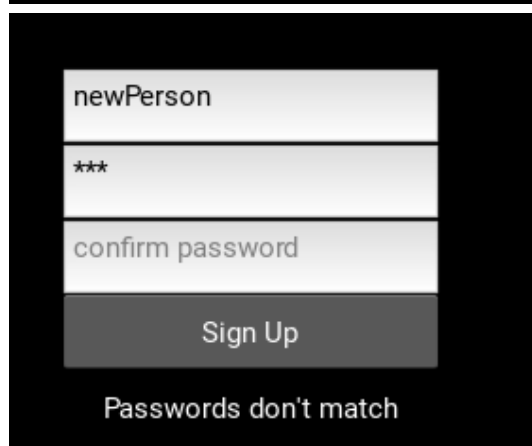
Login and signup has been built to satisfy the [success criteria](#) and as designed, on login you get put into the home page. On a successful signup you will go to the login page, you will not be logged in automatically. I made it so you are not logged in automatically as you should be able to gain limited guest access without an account, so being logged in automatically will prevent you from choosing to do this.

The [data validation](#) and [verification](#) as outlined in those sections have been implemented.

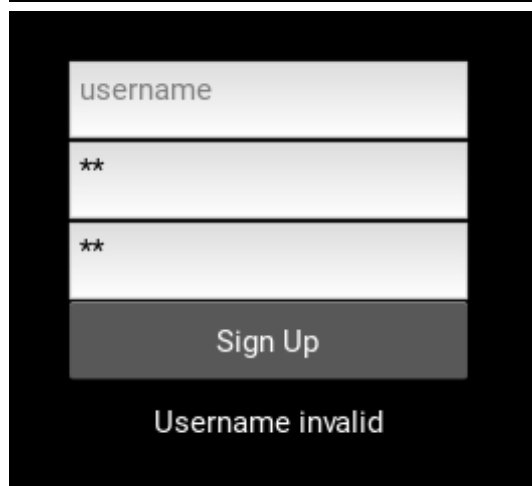
valid as well as getting you to confirm it)



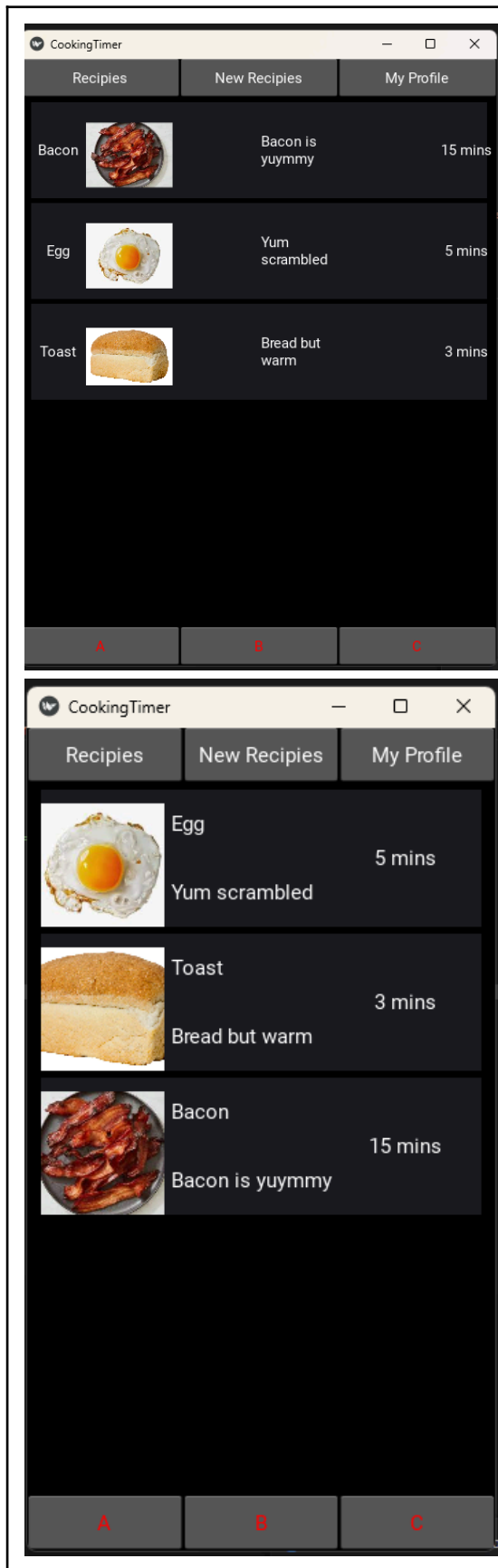
A sign-up form with a black background. It contains three white input fields: the first contains 'bob', the second is labeled 'password', and the third is labeled 'confirm password'. Below the fields is a grey 'Sign Up' button. At the bottom, the text 'Username already taken' is displayed in white.



A sign-up form with a black background. It contains three white input fields: the first contains 'newPerson', the second contains '***', and the third is labeled 'confirm password'. Below the fields is a grey 'Sign Up' button. At the bottom, the text 'Passwords don't match' is displayed in white.



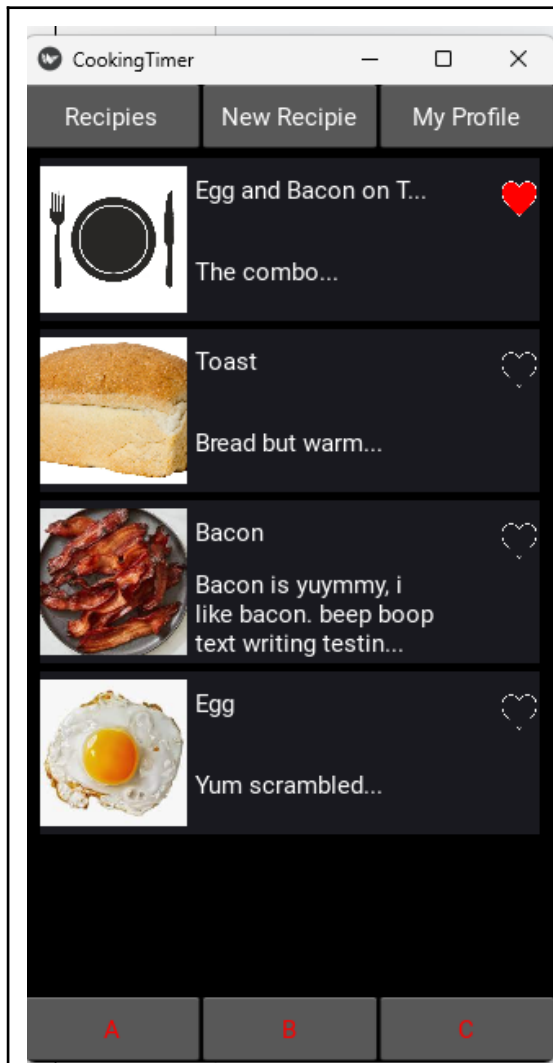
A sign-up form with a black background. It contains three white input fields: the first is labeled 'username', the second contains '**', and the third contains '**'. Below the fields is a grey 'Sign Up' button. At the bottom, the text 'Username invalid' is displayed in white.



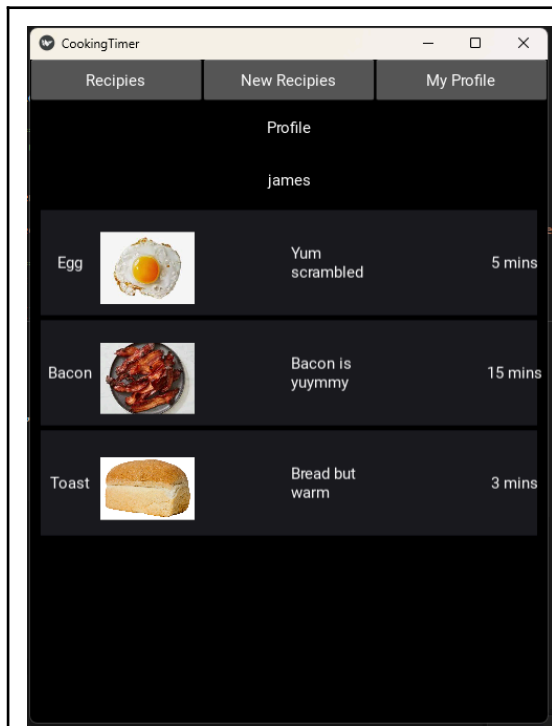
There has also been another update to the interface to make it more accessible and intuitive.

There is now a like button allowing for the user to toggle the liking of a recipe and viewing its current state. This was done to make it quick and easy to see recipes you have already liked and to like recipes without opening the full page.

The text position and colour has been changed, the text is now white making it stand out more and there is less wasted space so the images and text are bigger and easier to see /read.



Improved visuals
Added like button (saves recipes to your profile to view later)

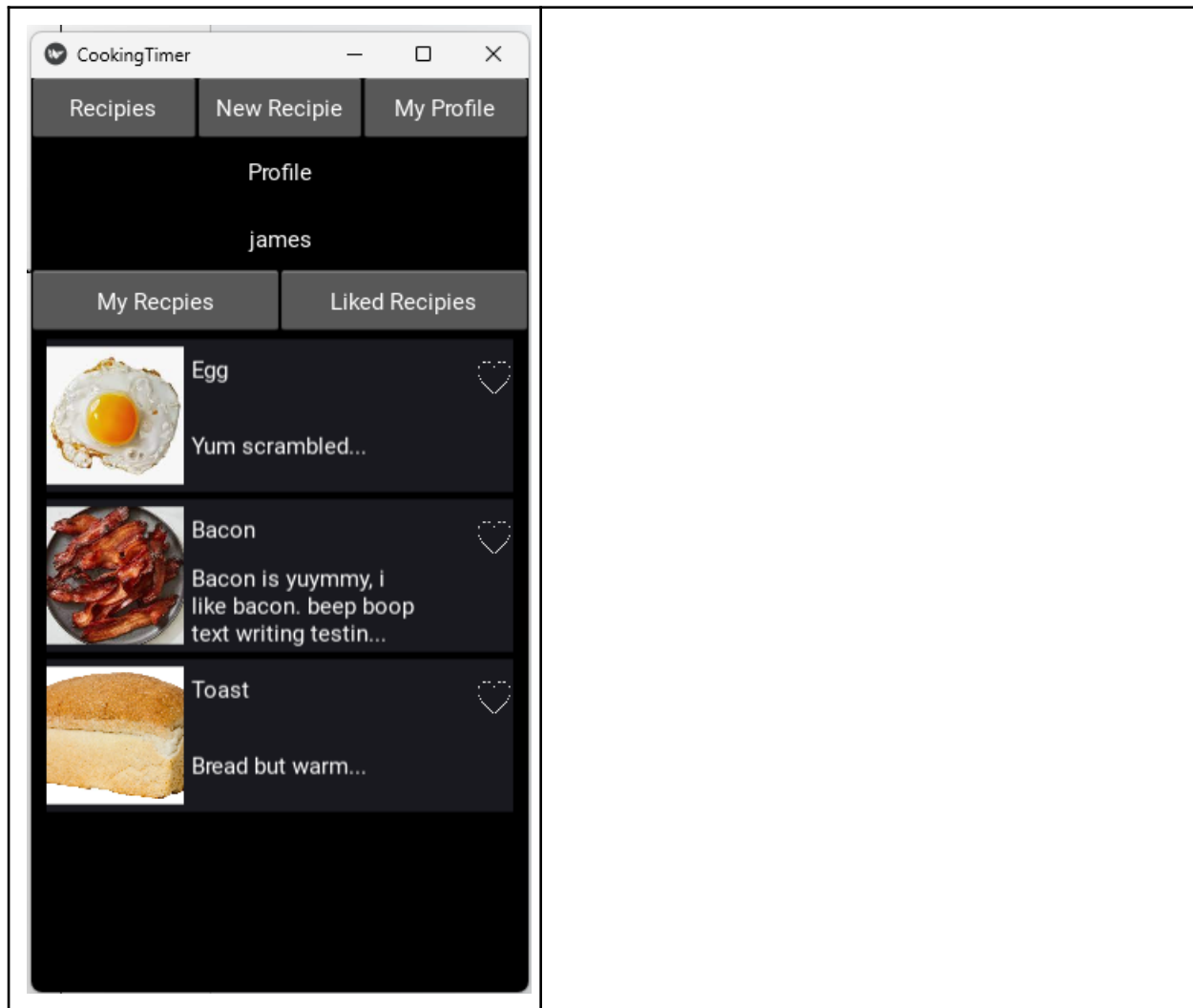


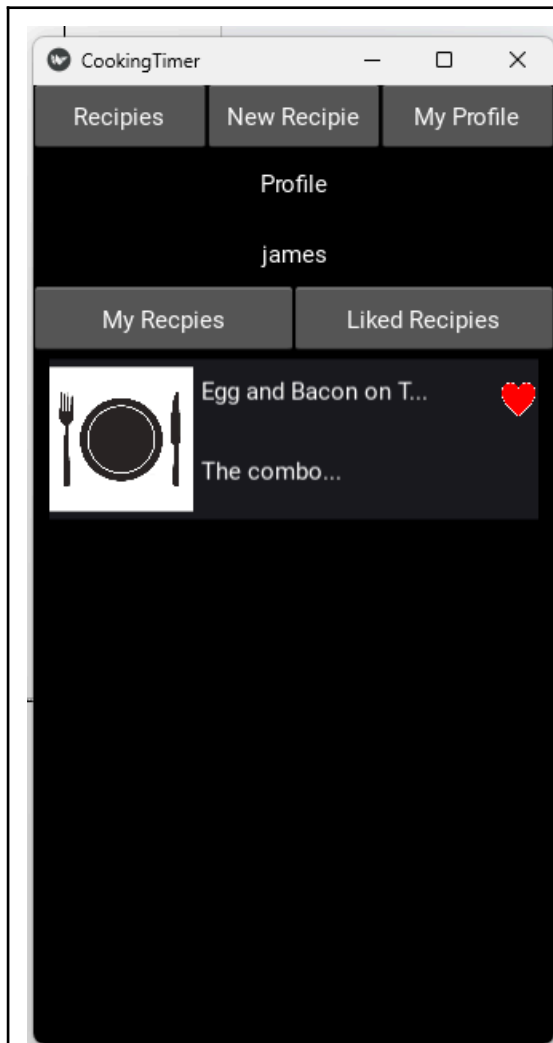
Semi-working profile feature, allows you to view their description and recipes they have made but no way of accessing profiles other than your own.

A profile view feature has been added, initially there were issues loading the users recipes but it was fixed by adding some filter features to getRecipes function. This will let you see the recipes the user has made and if you have permission their liked recipe, due to the scope of this project the only the user can see their own liked recipes but if the project were to continue a friend/follow system would be implemented which would let you see other peoples liked recipes.

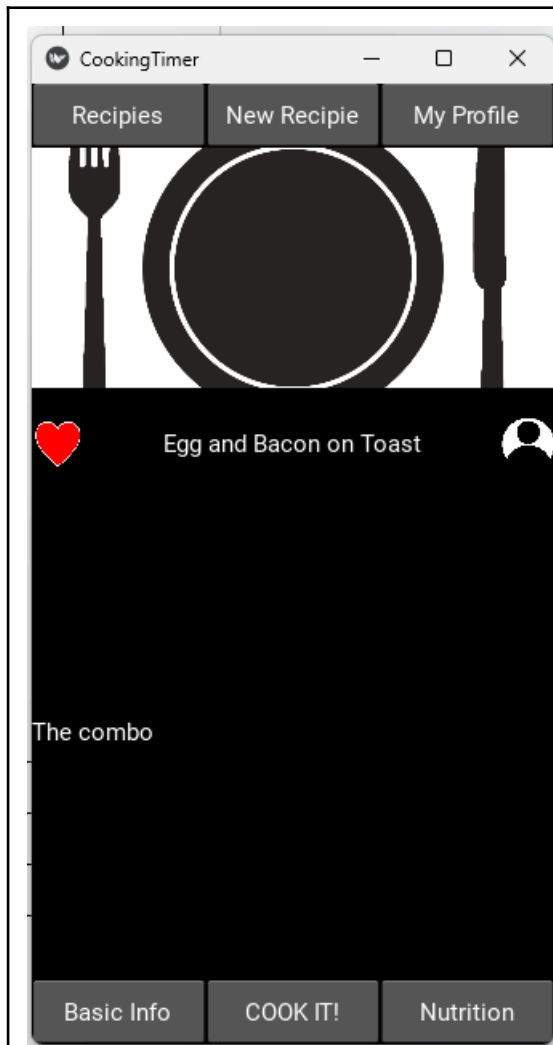
Letting users see their own liked recipes on their profile will make it quick and easy to find them without having to randomly stumble on them in the home page, the point of the like feature was to act partially as a saving/bookmark feature for the recipes. This is talked about in the [interface design](#).

I have also added a recipe view feature, this will allow for the user to open up recipes into a different interface where they can see and access a variety of features, such as: a full description of the recipe, link to the profile view feature, a like button like the one seen on the banner, access to nutritional information and access to the timer feature. This feature is needed as it abstracts visual clutter of the home screen, otherwise there would be too much information on the banners. Having all this information on the banners would mean it would either have to be so compact it would be hard to read the text or it would have to be so big, the home screen no longer serves its purpose as a quick recipe browsing interface.





Default image for recipes if none
provided
View recipe page



Can see recipe makers profile

Recipies	New Recipie	My Profile
Active Process		
5		
IDs of proccess which need to finish before this starts (seperate by new lines) or leave blank if none (only one can be left blank and it will be the first thing in the recipie)		
Prepare carrots, potatos		
1		
Passive Process		
20		
0		
Put carrots and potatos in oven		
Basic Info	Timings	Finish

The recipe making feature has been built, it allows the user to build recipes with timings and add them to the database.

The feature is split into 3 interfaces as defined by the [success criteria](#).

The first page lets the user input the basic info, recipe name and a short description.

The 'Timings' interface allows users to set up the timer for the app, it is designed to be as intuitive as possible in the scope of the project. If there was more time I would implement an interactive graph or Gantt chart to build the timer, this would be more intuitive and easy to understand as it is a familiar format to people. The interface now is simple and easy to use however it isn't immediately clear what the user should do due to its lack of similarity with other parts of the app and other applications.

The final screen just runs checks to make sure the recipe is valid, then generates the network and saves it to the database. This is easy to use and it is only 2 clearly labelled buttons.

Recipies

New Recipie

My Profile

Recipie Name

Description

Basic Info

Timings

Finish

[illegible]

Calculation of critical path working, allows for gantt chart generation and optimisation

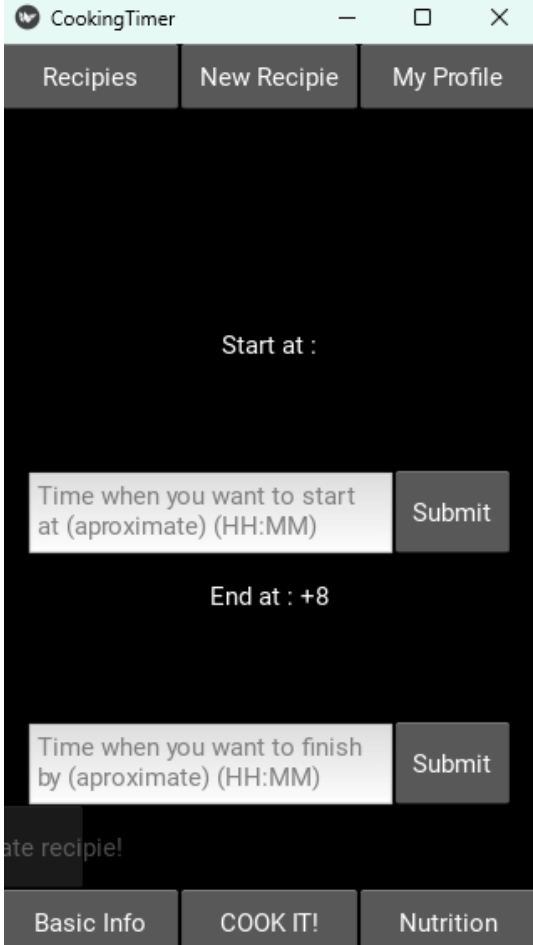
BUG Off by one error

```
2
False
{'start': ['0', '1'], '0': ['1', '2'], '1': ['1', '3']}
##
['1']
[]
['3']
['0', '1']
[]
```

EXPECTED A 'H' as output, line 7 will trigger a for loop shown below. As the H was not output, it shows the for loop did not run

The system to check if a recipe is valid and to generate the network is working, it is built into the final screen of the recipe maker interface. The way it works is described in the [Network Generation Algorithm](#) section.

I split up the system into two parts, checking if the recipe is valid and then making it. This is necessary for a couple of reasons: it means the user can check if the recipe is valid before committing it to be added to the database; the other reason is if the the checking and building was done separate to the submitting it to the database then it would waste computational resources as it isn't necessary unless its saved. Splitting the system up and doing it in this way saves computational resources whilst adding functionality to the recipe making system.

<pre>print("##") print(sharedDependency) print(listA) print(listB) #for each item in listB: for j in range(0, len(listB) - 1): print("H") #add a dummy to tmpDummy nodeOrigin = network.get(j)</pre>	During development there was an off by one error, this bug was difficult to detect due to the size of the algorithm and its nature of not crashing the code and making only small errors in the output. It did one iteration less of adding dummy nodes to the network due to this making some recipes miss data in a way in which made them still work but not act as the creator intended. This needed to be fixed as it could result in some recipes not enforcing one part be done before another, e.g. it could mean cooking the potatoes is done before preparing them.
	The timer feature has been started, the start/end recipe can be set using the first screen, I have added labels and text in the boxes to help make it more intuitive to the user, this is necessary as the feature is sensitive to the syntax of the input so it needs to be explained to the user in a clear way. It then loads the recipe from the database and displays the shortest path at the top. Due to the limited time for the project that is all I accomplished, if the project were to continue, the rest of the recipe would be loaded onto the interface. It would be shifted according to the start/end time input at the start. The 'to do' and 'on going' sections would be added and the timing adjustment system would be built to adjust the gantt chart dynamically depending on when different parts of the recipe start/end.

CookingTimer

Recipies

New Recipie

My Profile

Start at :

23:02

Submit

End at : +8

23:10

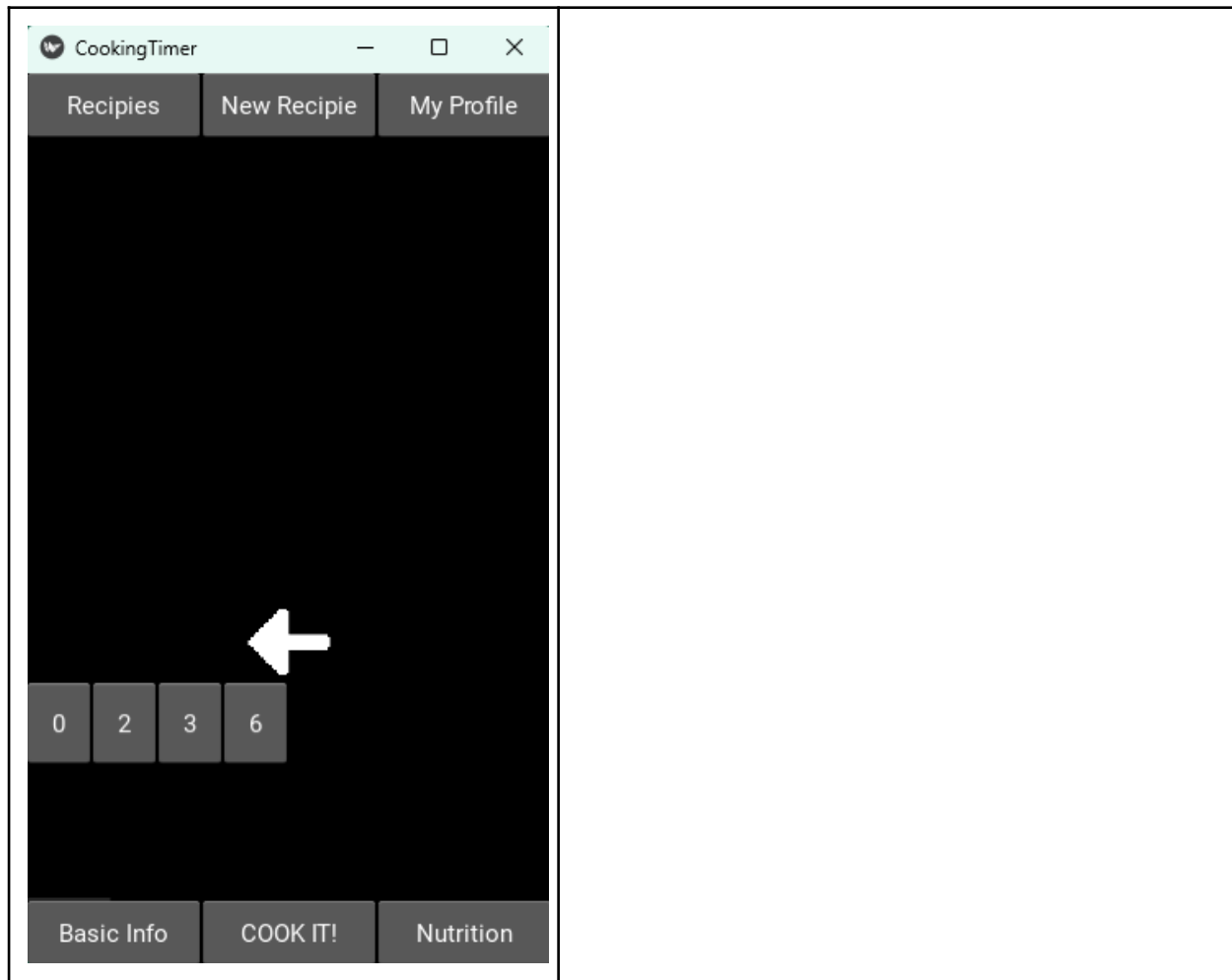
Submit

ate recipie!

Basic Info

COOK IT!

Nutrition

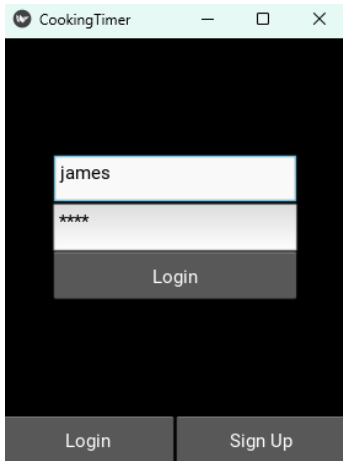
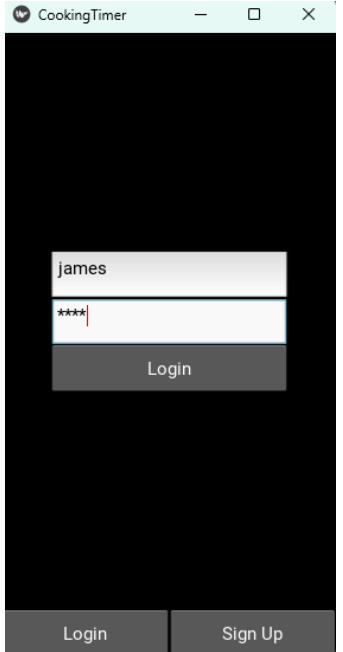


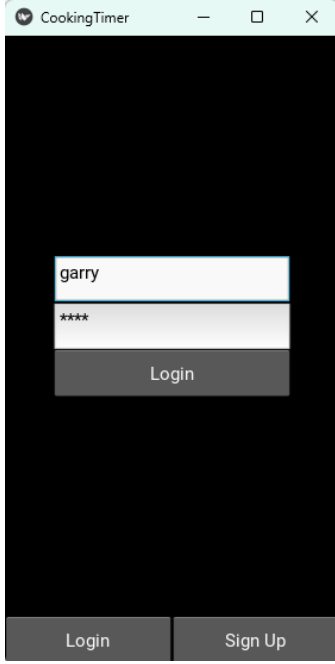
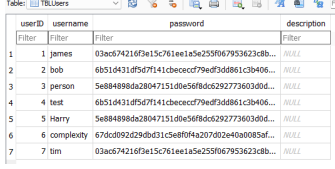
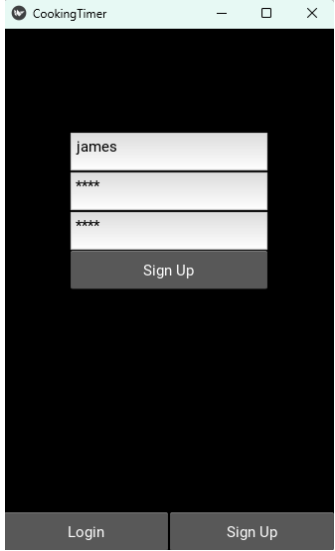
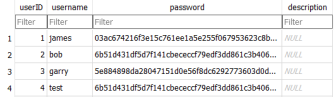
Testing to inform development

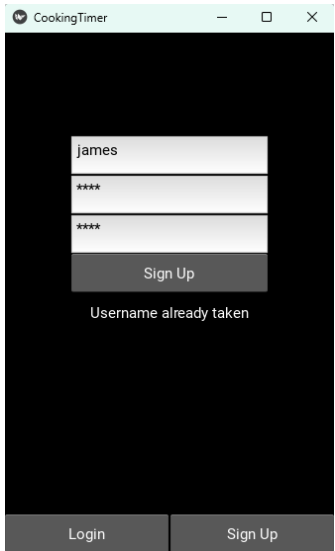
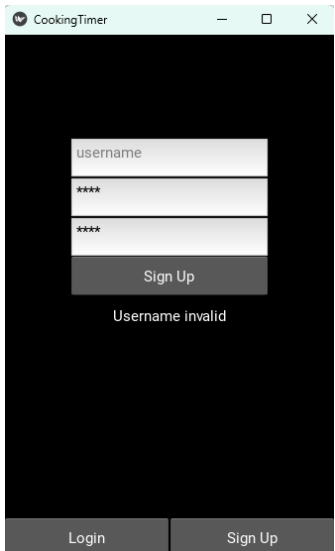
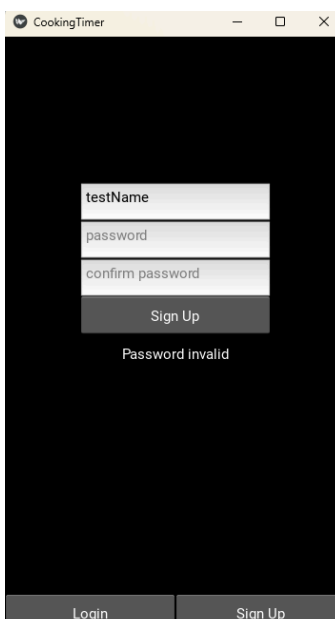
Incremental testing is shown in the previous section of the document under [development and testing](#).

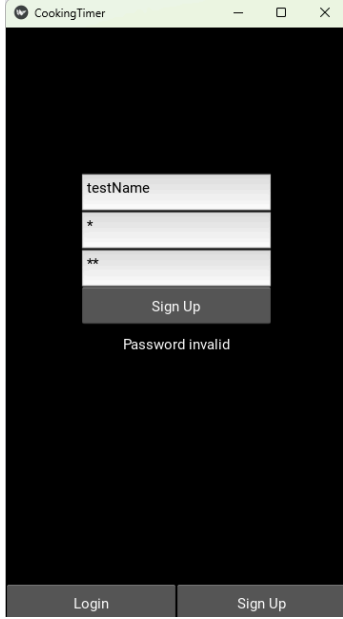
See [Test Data](#)

Algorithm	Type	Test Data	Expected output	Evidence and notes	Successful?
-----------	------	-----------	-----------------	--------------------	-------------

SubmitLoginDetails	Normal	['James', '1234']	Successful login	 Here 'James' exists and the correct password was input. Login was successful.	Yes
	Normal	['James', 'abcd']	Failed login	 Here 'James' exists and the incorrect password was input. Login was not successful.	Yes

	Normal	['Garry', '1234']	Failed login	  Here 'Garry' does not exist. Login was not successful.	Yes
SubmitSignUpDetails	Normal	['James', '1234', '1234']	Successful signup	 	Yes

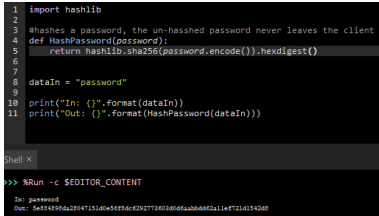
	Normal	['James', '1234', '1234']	"Username already exists"		Yes
	Erroneous	['', '1234', '1234']	"Invalid username"		Yes
	Erroneous	['James', '', '']	"Invalid password"		Yes

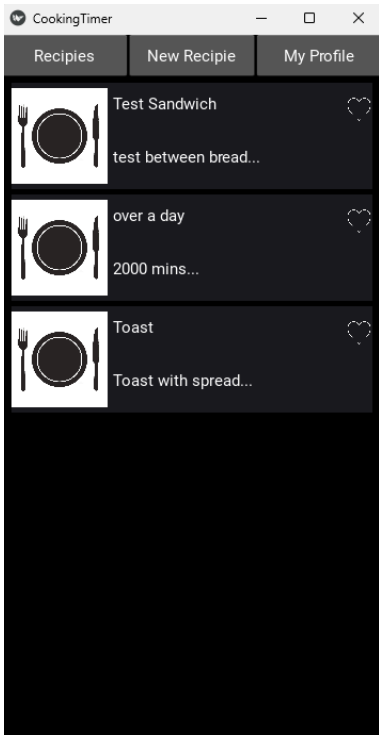
	Erroneous	['James', '1234', 'abcd']	"Passwords don't match"		Yes
Calculate DependencyTrees	Normal	[[{'id': '0', 'activePassive': '0', 'aproxTime': '5', 'conditions': '', 'description': ''}]]	[[{'0': ['start'], 'start': [], 'end': ['0']], {'0': ['end'], 'start': ['0'], 'end': [], ['0'], ['0']}]	[[{'0': ['start'], 'start': [], 'end': ['0']], {'0': ['end'], 'start': ['0'], 'end': [], ['0'], ['0']}]	Yes
	Normal	[[{'id': '0', 'activePassive': '0', 'aproxTime': '5', 'conditions': '', 'description': ''}, {'id': '1', 'activePassive': '0', 'aproxTime': '5', 'conditions': '', 'description': ''}, {'id': '2', 'activePassive': '0', 'aproxTime': '5', 'conditions': '0\n1', 'description': ''}, {'id': '3', 'activePassive': '0', 'aproxTime': '5', 'conditions': '1', 'description': ''}]]	[[{'0': ['start'], '1': ['start'], '2': ['0', '1'], '3': ['1'], 'start': [], 'end': ['2', '3']], {'0': ['2'], '1': ['2', '3'], '2': ['end'], '3': ['end'], 'start': ['0', '1'], 'end': [], ['0', '1'], ['2', '3']}]	[[{'0': ['start'], '1': ['start'], '2': ['0', '1'], '3': ['1'], 'start': [], 'end': ['2', '3']], {'0': ['2'], '1': ['2', '3'], '2': ['end'], '3': ['end'], 'start': ['0', '1'], 'end': [], ['0', '1'], ['2', '3']}]	Yes
	Erroneous	[[{'id': '0', 'activePassive': '0', 'aproxTime': '2', 'conditions': '1', 'description': ''}, {'id': '1', 'activePassive': '0', 'aproxTime': '2', 'conditions': '0', 'description': ''}]]	-1 / None	-1 / None	Yes

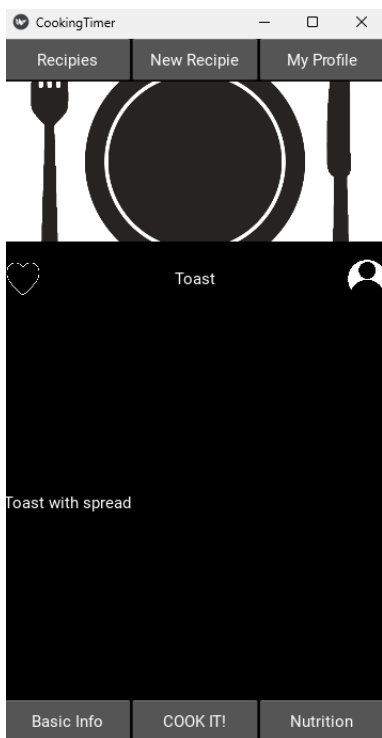
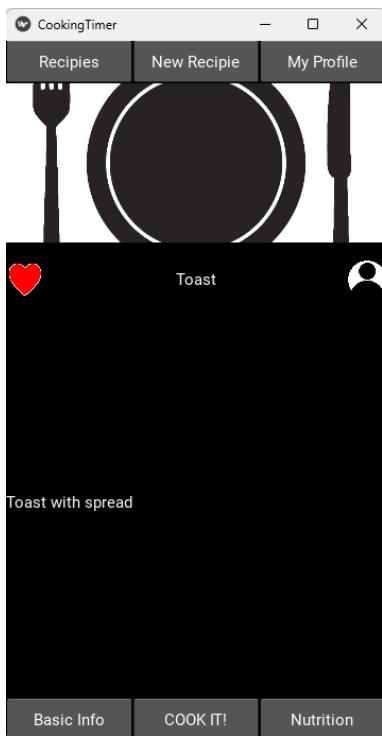
Traverse Process esBuildN etwork	Normal	{'0': ['start'], '1': ['0'], 'start': [], 'end': ['1']}, {'0': ['1'], '1': ['end'], 'start': ['0'], 'end': [], ['0'], ['1']}	{"network": {"start": ["0", "1"], "0": ["1", "0": ["1", "2"], "end": ["2", "3"]}, "nodes": {"1": ["start", "2": ["0", "3": ["end"]], "criticalPath": ["start", "0", "end"], "earlyLateTimes": {"0": [0, 0], "3": [5, 5], "1": [0, 0], "2": [5, 5]}, "activeProcesses": []}}	Yes	
	Normal	{'0': ['start'], '1': ['start'], '2': ['0', '1'], '3': ['1'], 'start': [], 'end': ['2', '3']}, {'0': ['2'], '1': ['2', '3'], '2': ['end'], '3': ['end'], 'start': ['0', '1'], 'end': [], ['0', '1'], ['2', '3']}	{"network": {"start": ["0", "1"], "0": ["1", "0": ["1", "2"], "1": ["1", "3"], "dummy0": ["3", "2"], "2": ["2", "4"], "3": ["3", "4"], "end": ["4", "6"]}, "nodes": {"1": ["start", "2": ["1", "0", "dummy0"], "3": ["1", "4": ["2", "3"], "6": ["end"]], "criticalPath": ["start", "1", "2", "end"], "earlyLateTimes": {"0": [0, 0], "6": [10, 10], "1": [0, 0], "2": [5, 5], "4": [10, 10], "3": [5, 5]}, "activeProcesses": []}}	Yes	
TimerInita lTiming sSetup	Normal	[5, 0, '0:0']	['00:00', '00:05']	['00:00', '00:05']	Yes
	Normal	[65, 0, '0:0']	['00:00', '01:05']	['00:00', '01:05']	Yes
	Errone ous	[2000, 0, '0:0']	Throws error, recipe time should not be saved if over one day (discussed in limitations of computational solution)	['00:00', '33:20']	No See maintaining the application

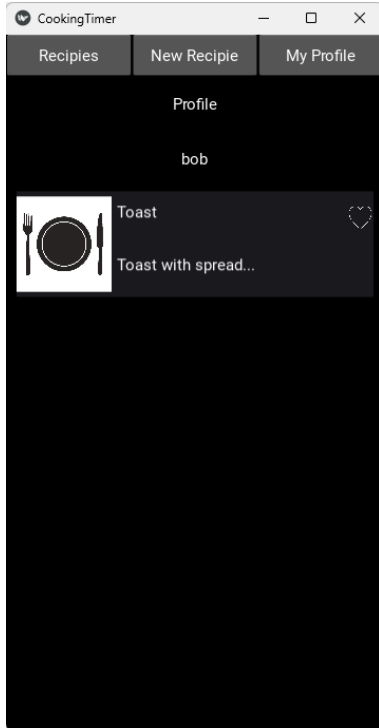
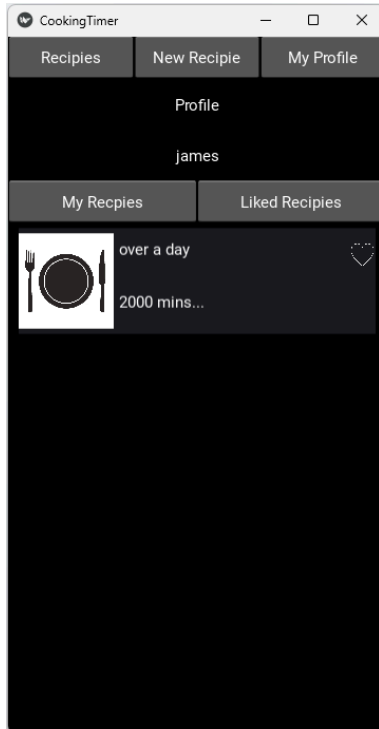
Evaluation

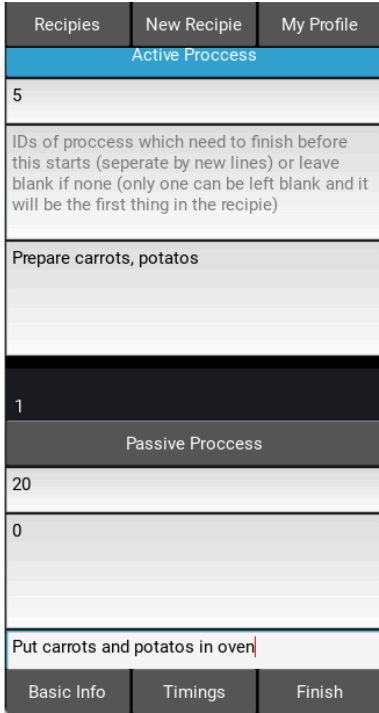
Testing against success criteria

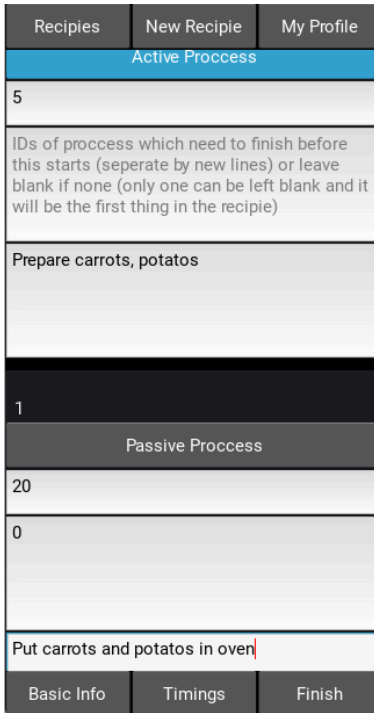
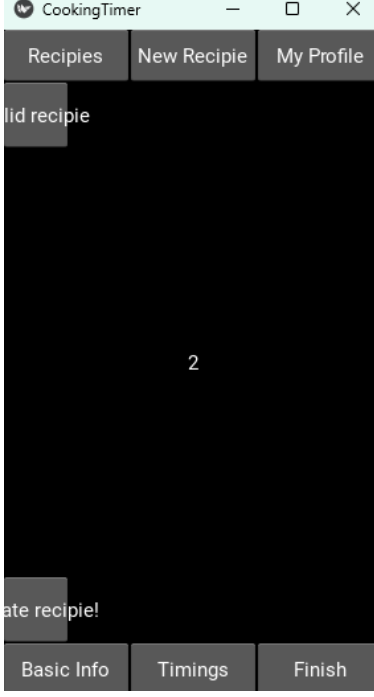
Criteria	Proof	Notes	Successful/ unsuccessful/ Partially Successful
The user should be able to log into their account	See Testing to inform development Clicking the login button will lead to the home screen where user information passes verification.	The design for the login system worked well, no major changes were made from the original design.	Successful
The user should be able to create an account	See Testing to inform development Clicking the signup button will create an account and lead to the login button where the user information passes validation and verification.		Successful
The usernames can only be used once and should not be null	See AddUser() in databaseManager.py line 209 See SubmitSignUpDetails() in main.py line 82		Successful
Passwords should not be null	See SubmitSignUpDetails() in main.py line 90		Successful
Passwords are hashed	See SignUp() in userManager.py line 23  The screenshot shows a Python script snippet for password hashing using hashlib. It defines a function HashPassword that takes a password and returns its SHA256 hash. Below the code, a terminal window shows the execution of the script, displaying the input password and its corresponding hashed output.	This system worked as originally designed however it didn't act as proper security and could be bypassed in deployment. This is talked about in Maintaining the application .	Partially Successful

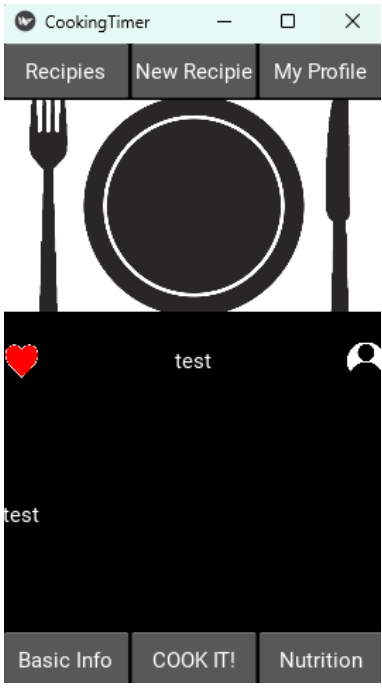
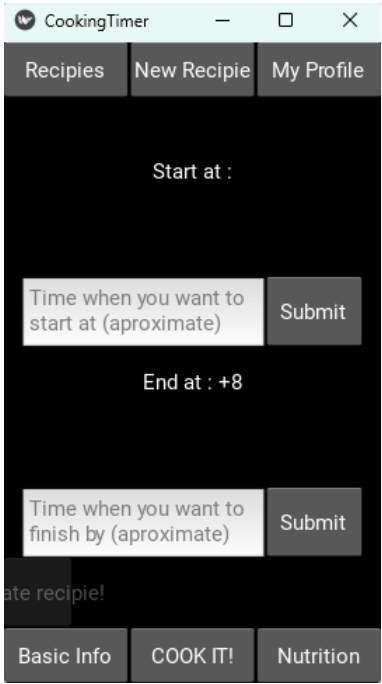
After logging in the user should be able to see a list of recipes from the database		This worked well, only the interface was changed from the design, this was to remove visual clutter to make it easier to read. It also is easier to develop in a consistent style using the Kivy system.	Successful
Each recipe should be shown in a recipe card	See above		Successful
Each recipe card will show the name of the recipe, any related info and whether or not the user has liked that recipe.	See above		Successful

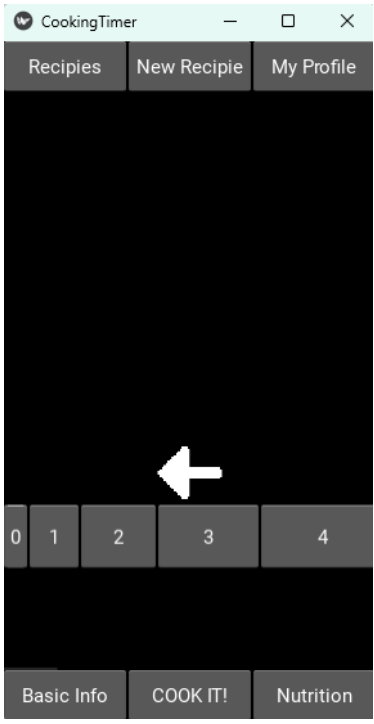
When a 'recipe card' is clicked on it should take the user to the recipe page			Successful																																										
The like button should be able to be toggled and save its state to the server	 <table><tr><th></th><th>userRecipeID</th><th>userID</th><th>recipeID</th><th>owner</th><th>liked</th></tr><tr><th></th><th>Filter</th><th>Filter</th><th>Filter</th><th>Filter</th><th>Filter</th></tr><tr><td>1</td><td>1</td><td>2</td><td>1</td><td>1</td><td>1</td></tr><tr><td>2</td><td>2</td><td>4</td><td>1</td><td>0</td><td>1</td></tr><tr><td>3</td><td>3</td><td>4</td><td>2</td><td>1</td><td>0</td></tr><tr><td>4</td><td>4</td><td>1</td><td>3</td><td>1</td><td>0</td></tr><tr><td>5</td><td>5</td><td>1</td><td>1</td><td>0</td><td>0</td></tr></table>		userRecipeID	userID	recipeID	owner	liked		Filter	Filter	Filter	Filter	Filter	1	1	2	1	1	1	2	2	4	1	0	1	3	3	4	2	1	0	4	4	1	3	1	0	5	5	1	1	0	0	The original database design did not allow for liking to be implemented, for it to work i updated TBL_UserRecipes to include a liked option which stored if a user liked a recipes.	Successful
	userRecipeID	userID	recipeID	owner	liked																																								
	Filter	Filter	Filter	Filter	Filter																																								
1	1	2	1	1	1																																								
2	2	4	1	0	1																																								
3	3	4	2	1	0																																								
4	4	1	3	1	0																																								
5	5	1	1	0	0																																								

	<table><thead><tr><th></th><th>userRecipeID</th><th>userID</th><th>recipeID</th><th>owner</th><th>liked</th></tr><tr><th></th><th>Filter</th><th>Filter</th><th>Filter</th><th>Filter</th><th>Filter</th></tr></thead><tbody><tr><td>1</td><td>1</td><td>2</td><td>1</td><td>1</td><td>1</td></tr><tr><td>2</td><td>2</td><td>4</td><td>1</td><td>0</td><td>1</td></tr><tr><td>3</td><td>3</td><td>4</td><td>2</td><td>1</td><td>0</td></tr><tr><td>4</td><td>4</td><td>1</td><td>3</td><td>1</td><td>0</td></tr><tr><td>5</td><td>5</td><td>1</td><td>1</td><td>0</td><td>1</td></tr></tbody></table> See row 5		userRecipeID	userID	recipeID	owner	liked		Filter	Filter	Filter	Filter	Filter	1	1	2	1	1	1	2	2	4	1	0	1	3	3	4	2	1	0	4	4	1	3	1	0	5	5	1	1	0	1		
	userRecipeID	userID	recipeID	owner	liked																																								
	Filter	Filter	Filter	Filter	Filter																																								
1	1	2	1	1	1																																								
2	2	4	1	0	1																																								
3	3	4	2	1	0																																								
4	4	1	3	1	0																																								
5	5	1	1	0	1																																								
The profile view button should take the user to the profile of the user who made the recipe when clicked. This page should also be accessible through a 'My profile' tab.		This system worked well and no major changes from the design were made.	Successful																																										
The profile view should show liked recipes if the profile is of the user currently logged in			Successful																																										

The nutrition button on the recipe view interface should give nutritional information in the recipe.		The ingredient and nutrition system was not built, this is talked about in maintaining the application and evaluating features	Unsuccessful
The recipe making interface should have 3 tabs for: general details, recipe processes and then uploading to database			Successful
The recipe making interface should allow the user to input a name, basic description in the general details section			Successful

The processes interface should let the user input data about timing, dependencies and methods.		This system worked well as it was designed, however the active/passive process feature wasn't needed due to the incomplete timer feature.	Successful
There should be a place to add ingredients to the recipe which links to a database of ingredients.		The ingredient and nutrition system was not built, this is talked about in maintaining the application and evaluating features	Unsuccessful
The final screen of the recipe making interface should allow the user to check if the recipe is valid and then save it to the database.		This system works but the interface was not completed, this is discussed more in evaluating usability.	Partially Successful

There should be a button which lets the user open the timer interface for the recipe.	 The screenshot shows the 'CookingTimer' app window. At the top, there are three tabs: 'Recipies', 'New Recipie', and 'My Profile'. Below the tabs is a large image of a plate with a fork and knife. Underneath the image, there's a red heart icon, the word 'test', and a profile icon. At the bottom, there are three buttons: 'Basic Info', 'COOK IT!', and 'Nutrition'.		Successful
The app should let the user set a start/end time for the recipe.	 The screenshot shows the 'CookingTimer' app window. At the top, there are three tabs: 'Recipies', 'New Recipie', and 'My Profile'. Below the tabs, the text 'Start at :' is displayed. Underneath, there is a text input field with the placeholder 'Time when you want to start at (aproximate)' and a 'Submit' button. Below that, the text 'End at : +8' is displayed. Underneath, there is another text input field with the placeholder 'Time when you want to finish by (aproximate)' and a 'Submit' button. At the bottom, there are three buttons: 'Basic Info', 'COOK IT!', and 'Nutrition'.		Successful

The app should generate a gantt chart timer from the network and nodes data		The timer interface was not successful, this is talked about in maintaining the application and evaluating features	Unsuccessful
The timer interface should show upcoming and ongoing processes in labelled boxes		The timer interface was not successful, this is talked about in maintaining the application and evaluating features	Unsuccessful
The timer interface should adapt to how long different processes take		The timer interface was not successful, this is talked about in maintaining the application and evaluating features	Unsuccessful

Evaluating Usability

Overall the usability of the application has been successful, the interface design is consistent, buttons are clearly labelled and/or use an appropriate symbol, there are prompts to help the user understand how to use the application (like in the recipe making interface) and it is laid out in a clear way.

The theme of the interface (black background, white text and grey highlights) (see screenshots in [testing against success criteria](#)) works well as it makes the text easy to read and helps make important things (like the buttons) stand out. It is also good as coloured things stand out even more (such as buttons or images) allowing for the user to immediately know what is important, making the application intuitive.

The use of text and prompts also allows the user to understand what things do if it's not conveyed clearly by the interface.

A few things have not been completed due to the time constraints of the project, namely the final screen of the recipe making interface and the timer interface. Both interfaces have formatting issues, making them unintuitive due to things like text or other elements clipping through or being completely off screen and lack of labels and prompts. See [Testing against Success Criteria](#). If I were to continue I would solve this, I talk about this in [Maintaining the application](#).

Evaluating Features

The success of features in the app has been good, most feature work as defined by the success criteria, accounting for erroneous and boundary input and any algorithms behind the features are trackable.

The network building algorithm has been successful. It works for any valid input, it can account for boundary, erroneous and normal data and can deal with any number or complexity of a network.

The password system has been partially successful, there were some issues which are outlined in [Maintaining the application](#)

The timer system was not successful, it was incomplete due to the time constraints of the project, what would happen to it if the project were to continue is outlined in the [maintaining the application](#) section.

The ingredients and nutrition systems were unsuccessful, they were not implemented fully due to the time limitations of the project. There is some framework for them in the database and interface but they do not work as defined by the [success criteria](#) at all.

Maintaining the application

Overall, the project was successful, but due to the time limitations of this project (such as time) and the limitations of my solution, I could not build the application to solve the initial problem (see [overview](#)) in a way which could be deployed straight away. If the project were to continue and then be deployed, I would have to fix/add some more features and set up maintenance to keep the application working. This is necessary as otherwise the application would not build a user base due to its missing features and sometimes un-intuitive interfaces, without a user base the application will not be successful.

The current solution is a good start but, for the application to be deployed, some limitations of the solution would need to be overcome. It is currently a local application, it has only a few features, and due to the time limitations of the project, it doesn't fully meet the success criteria. The lack of features could be solved by adding things such as recipe recommendation algorithms, a proper liking/review system and user following system. These features could work together, higher rated recipes and ones made by higher followed users would be shown more allowing for the most popular recipes to be shown more.

A feature that allows recipe recommendations (such as a recommendation algorithm or filters) is necessary as right now it is random making it less accessible to people with dietary restrictions. As well as this, without it the app would not facilitate people's tastes and preferences making it less user friendly.

To solve the issue of the project not meeting the success criteria; the timer, ingredients and database system would have to be completed. Right now the timer is incomplete, missing most of its functionality as described in the [success criteria](#). With the timer I would have to fix the initial time setting feature which failed the tests, the system should be reworked to work for any length recipe starting at any timer and account for multiple days. This is necessary for two reasons, one being it currently does not work for recipes that go overnight which is a fairly common thing. The second reason is that it is hard to predict what the user will input and so it is sensible to build a robust system that will work for any input and not break.

There is also not currently an ingredients and nutrition system, and these systems would have to be added as without them, the application would not solve the initial problem of needing an app to aid people whilst cooking with both a timer and recipe sharing feature, see problem [overview](#).

After these features were added, I would need to redesign the database to be more robust and allow for more functionality, this is talked about in the [database design](#). The [databaseManager.py](#) would also need to be designed to work with the updated database and also to work on a server as it doesn't currently run on one. This is necessary as otherwise users would not be able to easily share recipes with each other. With this a server would need to be set up and the security would need to be improved to secure people's data, such as the password system.

It doesn't currently provide proper security. The system could be bypassed by intercepting other people's login requests then altering the code to just send that pre-hashed password. For it to work properly the code should use a hash function to check it hasn't been altered and send it with the password. This would prevent people from modifying the code. The hash that is used to ensure the code isn't generated would have to take into account things other than the code (maybe ip address/mac address, date, version number) so it couldn't be saved and bypassed like the current password system. The extra bits of data (ip address/mac

address/date/version number) would have to be sent to the server or fetched by the server and then the server would have to calculate what the hash should be and compare them.

I would also improve usability features, adding a help feature on some of the more complex features such as timer interface and recipe making interface. This is necessary as without it is difficult to understand how to use the application, this will make it less accessible for the user. I will also fix formatting for some interfaces in the app, such as the final page of the recipe making feature.

After the application is deployed it will need to be maintained and updated, bugs and outdated systems would need to be fixed/replaced/updated. If things such as android were updated the app would need to be updated to work with the new version and the server would need to be updated to deal with scalability as it would not be able to deal with large quantities of users in its current state. These updates and changes would need to be deployed to the user, a system such as google play store would make this easy as it already has built in systems for this and it is accessible to all android phones.

Final Code

Python

main.py

```
1.
2. from kivy.app import App
3. from kivy.uix.widget import Widget
4. from kivy.uix.boxlayout import BoxLayout
5. from kivy.uix.scrollview import ScrollView
6. from kivy.core.window import Window
7. from kivy.clock import Clock
8.
9. from kivy.uix.button import Button, ButtonBehavior
10. from kivy.uix.label import Label
11. from kivy.uix.image import Image
12.
13. from kivy.metrics import dp
14. from kivy.properties import StringProperty
15.
16.
17. from random import randint
18.
19. import databaseManager, userManager, utils
20.
21.
22. class CookingTimerApp(App):
23.     def build(self):
24.         return CookingTimerInterface()
25.
```

```

26. class CookingTimerInterface(BoxLayout):
27.     def __init__(self, **kwargs):
28.         super().__init__(**kwargs)
29.         self.currentUser = []
30.
31.     def LoadPage(self, i):
32.         self.remove_widget(self.children[0])
33.
34.         tmp = None
35.
36.         if i==0: tmp = WelcomeScreenInterface()
37.
38.         if i==1: tmp = LoggedInInterface()
39.
40.         self.add_widget(tmp, index=1)
41.
42.
43. class WelcomeScreenInterface(BoxLayout):
44.     def __init__(self, **kwargs):
45.         super().__init__(**kwargs)
46.
47.     def LoadMenu(self, i):
48.         self.remove_widget(self.children[1])
49.
50.         tmp = None
51.
52.         if i==0: tmp = LoginInterface()
53.
54.         if i==1: tmp = SignupInterface()
55.
56.         self.add_widget(tmp, index=1)
57.
58.
59. class LoginInterface(BoxLayout):
60.     def __init__(self, **kwargs):
61.         super().__init__(**kwargs)
62.
63.     def SubmitLoginDetails(self):
64.         userInfo = userManager.Login(self.children[0].children[0].children[2].text,
        self.children[0].children[0].children[1].text)
65.
66.         if userInfo == -1:
67.             print("Failed login")
68.             return
69.
70.         app= App.get_running_app()
71.         app.root.currentUser = [userInfo,
        self.children[0].children[0].children[1].text]
72.         app.root.LoadPage(1)
73.
74.
75. class SignupInterface(BoxLayout):
76.     signinFeedbackText = StringProperty("")
77.     def __init__(self, **kwargs):
78.         super().__init__(**kwargs)
79.
80.     def SubmitSignUpDetails(self):
81.         self.signinFeedbackText = ""

```

```

82.         if self.children[0].children[0].children[4].text == "":
83.             self.signinFeedbackText = "Username invalid"
84.             return
85.
86.         if
            userManager.IsUsernameAvailable(self.children[0].children[0].children[4].text) ==
            False:
87.             self.signinFeedbackText = "Username already taken"
88.             return
89.
90.         if self.children[0].children[0].children[3].text == "":
91.             self.signinFeedbackText = "Password invalid"
92.             return
93.
94.         if self.children[0].children[0].children[2].text !=
            self.children[0].children[0].children[3].text:
95.             self.signinFeedbackText = "Passwords don't match"
96.             return
97.
98.         if userManager.SignUp(self.children[0].children[0].children[4].text,
            self.children[0].children[0].children[3].text):
99.             self.parent.LoadMenu(0)
100.
101.             self.signinFeedbackText = "Sign up failed, try again"
102.
103.
104.     class LoggedInInterface(BoxLayout):
105.         def LoadMenu(self, i, *args):
106.             self.remove_widget(self.children[0])
107.
108.             tmp = None
109.
110.             if i==0: tmp = RecipeSearchInterface()
111.
112.             if i==1: tmp = NewRecipeInterface()
113.
114.             if i==2: tmp = ProfileViewInterface(args[0]) #pass in a -1 to get current
            user profile
115.
116.             if i==3: tmp = RecipeViewInterface(args[0])
117.
118.             if i==4: tmp = TimerInterface(args[0], args[1])
119.
120.             self.add_widget(tmp)
121.
122.
123.     class RecipeSearchInterface(BoxLayout):
124.         def __init__(self, **kwargs):
125.             super().__init__(**kwargs)
126.
127.
128.     class RecipeSearchScrollInterface(ScrollView):
129.         def __init__(self, id=None, owned=True, liked=None, **kwargs):
130.             super().__init__(**kwargs)
131.
132.             self.userID = id
133.
134.             self.add_widget(RecipeSearchBoxInterface(self.userID, owned, liked))

```

```

135.
136.
137.     class RecipieSearchBoxInterface(BoxLayout):
138.         def __init__(self, userID, owned, liked, **kwargs):
139.             super().__init__(**kwargs)
140.
141.             self.recipieFeed = []
142.
143.             self.LoadRecipies(userID, owned, liked)
144.
145.             #loads 10 more recipies
146.         def LoadRecipies(self, userID, owned, liked):
147.             if userID == None:
148.                 tmp = databaseManager.RecNumbInTable("TBLRecipies")
149.                 quantityToLoad = tmp if (tmp ≤ 10) else 10
150.
151.                 #algarthm would generate recipies based on user but randomly generated
152.                 for this
153.                     self.recipieFeed = utils.randomNumbSeq(quantityToLoad, 1, tmp, False,
154. currentList=self.recipieFeed)
155.
156.                     for i in self.recipieFeed:
157.                         self.add_widget(RecipieWigitBanner(recipeID=i))
158.
159.             else:
160.                 tmp = databaseManager.GetUsersRecipiesFromID(userID, owned, liked)
161.
162.                 for i in tmp:
163.                     self.add_widget(RecipieWigitBanner(recipeID=i[1]))
164.
165.     class RecipieWigitBanner(ButtonBehavior, BoxLayout):
166.         def __init__(self, recipeID, **kwargs):
167.             super().__init__(**kwargs)
168.
169.             self.recipieInfo = databaseManager.GetRecipieFromID(recipeID)
170.             self.userRecipieInfo = databaseManager.GetUserRecipieFromID(recipeID)
171.
172.             imgPath = ''
173.             if self.recipieInfo[4] == "":
174.                 imgPath = "Databases/RecipieImageData/recipeThumbnailDefault.png"
175.             else:
176.                 imgPath = str("Databases/RecipieImageData/" +
177. str("{0}-{1}.{2}".format(self.userRecipieInfo[1], self.recipieInfo[0],
178. self.recipieInfo[4])))
179.
180.             self.add_widget(Image(source=imgPath, fit_mode="cover", size_hint=(None,
181. None), size=(dp(90), dp(90)), pos_hint={"center_y" : 0.5}))#image
182.
183.             tmp = BoxLayout(orientation="vertical")
184.             tmp.add_widget(Label(text=self.recipieInfo[1], size_hint=(None, 0.4),
185. width=dp(150), text_size = (dp(150), None), shorten=True, shorten_from="right"))#title
186.             tmp.add_widget(Label(text=str(self.recipieInfo[2][:60] + " ... "),
187. size_hint=(None, 0.6), width=dp(150), text_size = (dp(150), None)))#, shorten=True,
188. shorten_from="right"))#description
189.             self.add_widget(tmp)
190.
191.             tmp = LikeRecipieButton([self.recipieInfo, self.userRecipieInfo])

```

```

185.         tmp.size_hint= (None, None)
186.         tmp.size=(dp(25), dp(25))
187.         tmp.pos_hint={"center_y" : 0.75}
188.
189.         self.add_widget(tmp)
190.         #self.add_widget(Label(text=str(self.recipeInfo[3]) + " mins",
pos_hint={'right': 0, 'center_y': .5}))#aprox timings
191.
192.     def LoadRecipePage(self):
193.         app= App.get_running_app()
194.         app.root.children[0].LoadMenu(3, [self.recipeInfo, self.userRecipeInfo])
195.
196.
197.     class RecipeViewInterface(BoxLayout):
198.         def __init__(self, recipeInfo, **kwargs):
199.             super().__init__(**kwargs)
200.
201.             self.recipeInfo = recipeInfo
202.
203.             print(recipeInfo)
204.
205.             self.add_widget(RecipeViewInterfaceBasicInfo(self.recipeInfo), 1)
206.
207.     def LoadRecipeViewPage(self, i):
208.         self.remove_widget(self.children[1])
209.
210.         tmp = None
211.
212.         if i==0: tmp = RecipeViewInterfaceBasicInfo(self.recipeInfo)
213.
214.         if i==1: tmp = TimerInterface(self.recipeInfo)
215.
216.         #if i==2: tmp = ProfileViewInterface()
217.
218.         self.add_widget(tmp, 1)
219.
220.
221.     class RecipeViewInterfaceBasicInfo(BoxLayout):
222.         def __init__(self, recipeInfo, **kwargs):
223.             super().__init__(**kwargs)
224.
225.             self.recipeInfo = recipeInfo
226.
227.             imgPath = ''
228.             if recipeInfo[0][4] == "":
229.                 imgPath = "Databases/RecipeImageData/recipeThumbnailDefault.png"
230.             else:
231.                 imgPath = str("Databases/RecipeImageData/" +
str("{0}-{1}.{2}".format(recipeInfo[1][1], recipeInfo[0][0], recipeInfo[0][4])))
232.
233.             self.add_widget(Image(source=imgPath, fit_mode="cover", size_hint=(1,
None), height=dp(150)), 1)#image
234.
235.             tmp = BoxLayout(size_hint=(1, None), height=(dp(60)))
236.
237.             tmp.add_widget(LikeRecipeButton(recipeInfo))
238.             tmp.add_widget(Label(text=recipeInfo[0][1], size_hint=(0.8, None),
height=(dp(50))))#title

```

```

239.         tmp.add_widget(ProfileLogoButton(recipeInfo[1][1]))
240.
241.         self.add_widget(tmp)
242.
243.         self.add_widget(Label(text=recipeInfo[0][2], size_hint_x=None,
width=dp(200), text_size = (dp(200), None), shorten=True,
shorten_from="right"))#description
244.
245.
246.     class ProfileLogoButton(ButtonBehavior, Image):
247.         def __init__(self, userID, **kwargs):
248.             super().__init__(**kwargs)
249.
250.             self.userID = userID
251.
252.             self.source = "media/profileLogo.png"
253.             self.on_press=self.OpenUserProfile
254.             self.size_hint= (0.1, None)
255.             self.height=dp(50)
256.
257.         def OpenUserProfile(self):
258.             app= App.get_running_app()
259.             app.root.children[0].LoadMenu(2, self.userID)
260.
261.
262.     class LikeRecipeButton(ButtonBehavior, Image):
263.         def __init__(self, recipeInfo, **kwargs):
264.             super().__init__(**kwargs)
265.
266.             self.recipeID = recipeInfo[0][0]
267.
268.             app= App.get_running_app()
269.             userLikedThis = userManager.HasUserLikedRecipe(app.root.currentUser[0][0],
app.root.currentUser[1], recipeInfo[0][0])
270.             imgPath = "media/heartLikedLogo.png" if userLikedThis else
"media/heartUnlikedLogo.png"
271.             self.source=imgPath
272.
273.             self.on_press=self.LikeRecipe
274.             self.size_hint= (0.1, None)
275.             self.height=dp(50)
276.
277.         def LikeRecipe(self):
278.             app= App.get_running_app()
279.             userManager.UserLikeRecipeToggle(app.root.currentUser[0][0],
app.root.currentUser[1], self.recipeID)
280.
281.             userLikedThis = userManager.HasUserLikedRecipe(app.root.currentUser[0][0],
app.root.currentUser[1], self.recipeID)
282.             imgPath = "media/heartLikedLogo.png" if userLikedThis else
"media/heartUnlikedLogo.png"
283.             self.source=imgPath
284.
285.
286.     class NewRecipeInterface(BoxLayout):
287.         def __init__(self, **kwargs):
288.             super().__init__(**kwargs)
289.

```

```

290.         # { recipe name, description, aprox time }
291.         self.recipeBasicInfo = {}
292.
293.         # { {id, aproxTime, conditions, conditionOffset, description} }
294.         self.proccessTimings = []
295.
296.         self.add_widget(NewRecipeBasicInfo(self.recipeBasicInfo), 1)
297.
298.     def LoadRecipeMakingPage(self, i):
299.         tmp = self.children[1].ReturnInfo() #for recipe saved data between screens
300.
301.         self.recipeBasicInfo = tmp[0] if tmp[0] != None else self.recipeBasicInfo
302.         self.proccessTimings = tmp[1] if tmp[1] != None else self.proccessTimings
303.
304.         self.remove_widget(self.children[1])
305.
306.         tmp = None
307.
308.         if i==0: tmp = NewRecipeBasicInfo(self.recipeBasicInfo)
309.
310.         if i==1: tmp = NewRecipeTimingInfo(self.proccessTimings)
311.
312.         if i==2: tmp = NewRecipeConfirm(self.recipeBasicInfo,
313.                                         self.proccessTimings)
314.
315.         self.add_widget(tmp, 1)
316.
317.     class NewRecipeBasicInfo(BoxLayout):
318.         def __init__(self, recipeBasicInfo, **kwargs):
319.             super().__init__(**kwargs)
320.
321.             if recipeBasicInfo.get("recipeName") != None:
322.                 self.children[0].children[0].children[1].text =
323.                     recipeBasicInfo.get("recipeName")
324.             if recipeBasicInfo.get("description") != None:
325.                 self.children[0].children[0].children[0].text =
326.                     recipeBasicInfo.get("description")
327.
328.         def ReturnInfo(self): #for recipe saved data between screens
329.             return [{"recipeName" : self.children[0].children[0].children[1].text,
330.                     "description" : self.children[0].children[0].children[0].text},
331.                     None]
332.
333.     class NewRecipeTimingInfo(BoxLayout):
334.         def __init__(self, proccessTimings, **kwargs):
335.             super().__init__(**kwargs)
336.
337.             proccessList = self.children[0].children[0]
338.
339.             for proccess in proccessTimings:
340.                 proccessList.LoadProccess()
341.
342.                 tmp = proccessList.children[1]
343.
344.                 tmp.children[4].text="ID: " + str(proccess.get("id"))

```

```

344.         tmp.children[3].state="down" if process.get("activePassive")=="1" else
        "normal"
345.         tmp.children[2].text=process.get("aproxTime")
346.         tmp.children[1].text=process.get("conditions")
347.         tmp.children[0].text=process.get("description")
348.
349.         proccessList.currentID = len(proccessTimings)
350.
351.     def ReturnInfo(self):#for recipie saved data between screens
352.         rtn = []
353.
354.         proccessList = self.children[0].children[0]
355.         for newProccesses in reversed(proccessList.children[1:]):
356.
357.             rtn.append({
358.                 "id" : str(newProccesses.children[4].text).strip("ID: "),
359.                 "activePassive" : "1" if newProccesses.children[3].state=="down"
        else "0",
360.                 "aproxTime" : newProccesses.children[2].text,
361.                 "conditions" : newProccesses.children[1].text,
362.                 "description" : newProccesses.children[0].text,
363.             })
364.
365.         return [None, rtn]
366.
367.
368.     class NewRecipieProccessList(BoxLayout):
369.         def __init__(self, **kwargs):
370.             super().__init__(**kwargs)
371.
372.             self.currentID = 0
373.
374.         def NewProccess(self):
375.             tmp = NewRecipieProcessBox()
376.             tmp.children[4].text="ID: " + str(self.currentID)
377.             self.add_widget(tmp, 1)
378.
379.             self.currentID += 1
380.
381.         def LoadProccess(self):
382.             self.add_widget(NewRecipieProcessBox(), 1)
383.
384.
385.     class NewRecipieProcessBox(BoxLayout):
386.         def __init__(self, **kwargs):
387.             super().__init__(**kwargs)
388.
389.
390.
391.     class NewRecipieConfirm(BoxLayout):
392.         def __init__(self, recipieBasicInfo, proccessTimings, **kwargs):
393.             super().__init__(**kwargs)
394.
395.             self.data = [recipieBasicInfo, proccessTimings]
396.
397.         def CheckRecipie(self):
398.             #valid network structure
399.             print("~~~~~")

```



```

400.         print(self.data[1])
401.         print("~~~~~")
402.
403.         dependencyTrees, fbk = utils.CalculateDependencyTrees(self.data[1])
404.
405.         print("~~~~~")
406.         print(dependencyTrees)
407.         print(fbk)
408.         print("~~~~~")
409.
410.         #valid basic info
411.         validBasicInfo = True
412.         if self.data[0]["recipieName"] == "":
413.             validBasicInfo = False
414.             fbk = "You must have a recipie name"
415.
416.         if self.data[0]["description"] == "":
417.             validBasicInfo = False
418.             fbk = "You must have a description"
419.
420.         self.children[2].text = fbk
421.
422.         if dependencyTrees != -1 and validBasicInfo:
423.             jsonData, aproxTime = utils.CalculateTimeingNetwork(dependencyTrees,
self.data[1])
424.
425.             self.children[1].text = str(aproxTime)
426.             self.aproxTime = aproxTime
427.             self.jsonData = jsonData
428.
429.             print("~~~~~")
430.             print(jsonData)
431.             print("~~~~~")
432.
433.             self.children[0].disabled = False
434.
435.         def CreateRecipie(self):
436.             app= App.get_running_app()
437.             username = app.root.currentUser[0][1]
438.             password = app.root.currentUser[1]
439.
440.             databaseManager.AddRecipie(self.data[0]["recipieName"],
self.data[0]["description"], self.aproxTime, None, self.jsonData, username,
userManager.HashPassword(password))
441.
442.         def ReturnInfo(self):#for recipie saved data between screens
443.             return [None, None]
444.
445.
446.
447.
448.         class ProfileViewInterface(BoxLayout):
449.             def __init__(self, userID, **kwargs):
450.                 super().__init__(**kwargs)
451.
452.                 if userID == -1:
453.                     app= App.get_running_app()
454.                     userInfoToLoad = app.root.currentUser[0]

```

```

455.
456.         else:
457.             userInfoToLoad = databaseManager.UserFromUserID(userID)
458.
459.             self.add_widget(Label(text=userInfoToLoad[1], size_hint= [1, 0.1]))
460.             if userInfoToLoad[2] ≠ None: self.add_widget(Label(text=userInfoToLoad[2],
size_hint= [1, 1]))
461.
462.             if userID == -1:
463.                 tmp = BoxLayout(size_hint_y=None, height=dp(40))
464.
465.                 tmp.add_widget(Button(text="My Recpies", on_press=lambda *args:
self.SetRecipieMode("owned", userInfoToLoad), size_hint_y=None, height=dp(40)))
466.                 tmp.add_widget(Button(text="Liked Recipies", on_press=lambda *args:
self.SetRecipieMode("liked", userInfoToLoad), size_hint_y=None, height=dp(40)))
467.
468.                 self.add_widget(tmp)
469.
470.             self.add_widget(RecipieSearchScrollInterface(userInfoToLoad[0], owned=True,
liked=None))
471.
472.         def SetRecipieMode(self, mode, userInfoToLoad):
473.             if mode == "owned":
474.                 ownedD = True
475.                 likedD = None
476.             if mode == "liked":
477.                 ownedD = None
478.                 likedD = True
479.
480.             self.remove_widget(self.children[0])
481.             self.add_widget(RecipieSearchScrollInterface(userInfoToLoad[0],
owned=ownedD, liked=likedD))
482.
483.
484.         class TimerInterface(BoxLayout):
485.             def __init__(self, recipieData, **kwargs):
486.                 super().__init__(**kwargs)
487.
488.                 #recipieData = [recipieInfo(from db), userRecipieInfo(from db)]
489.                 self.recipieInfo = recipieData[0]
490.                 self.networkData = self.recipieInfo[5]
491.                 self.userRecipieInfo = recipieData[1]
492.
493.                 print(self.recipieInfo)
494.                 print(self.networkData)
495.                 print(self.userRecipieInfo)
496.
497.                 #startEndTime = [startTime, endTime]
498.
499.                 self.add_widget(TimerSetup(self.recipieInfo[3]))
500.
501.             def LoadTheTimer(self, startTimestr, endTimestr):
502.                 self.remove_widget(self.children[0])
503.
504.                 tmp = BoxLayout()
505.
506.                 tmp.add_widget(ExitTimerInterfaceButton())
507.

```

```

508.         self.add_widget(tmp)
509.
510.         self.add_widget(TheTimer(self.networkData, startTimestr, endTimestr))
511.         self.i = 0
512.
513.         Clock.schedule_interval(self.UpdateCurrentTimeIndicator, 0.05)
514.
515.         def UpdateCurrentTimeIndicator(self, dt):
516.             #dt is delta time
517.             self.i += 10 * dt
518.             self.children[0].children[0].canvas.after.children[1].pos = [self.i, 100]
519.
520.
521.         class TimerSetup(BoxLayout):
522.             def __init__(self, aproxTime, **kwargs):
523.                 self.aproxTime = aproxTime
524.                 self.aproxTimeStr = str(aproxTime)
525.
526.                 super().__init__(**kwargs)
527.
528.                 self.startTimestr = "00:00"
529.                 self.endTimestr = str('{:02d}:{:02d}'.format( (aproxTime // 60),
530. (aproxTime - (60 * (aproxTime // 60))) )
531.
532.                 def OnTextChange(self, i):
533.                     value = self.children[3 if i==0 else 1].children[1].text
534.                     if len(value.split(":")) != 2 or not value.split(":")[0].isdigit() or not
535. value.split(":")[1].isdigit():
536.                         self.children[0].disabled = True
537.                         return None
538.
539.                     if i == 0:
540.                         start = value
541.
542.                         end = str('{:02d}:{:02d}'.format( int(start.split(":")[0]) +
543. (self.aproxTime // 60), int(start.split(":")[1]) + (self.aproxTime - (60 *
544. (self.aproxTime // 60))) )
545.
546.                         self.children[1].children[1].text = end
547.
548.                         end2 = end
549.
550.                         end2Int = int(int(end2.split(":")[0])*60 + int(end2.split(":")[1]))
551.                         startInt = end2Int - self.aproxTime
552.                         start2 = str('{:02d}:{:02d}'.format( int(startInt // 60), (startInt -
553. (60 * (startInt // 60))) )
554.
555.                         self.children[3].children[1].text = start2
556.
557.                     if i == 1:
558.                         end2 = value
559.
560.                         end2Int = int(int(end2.split(":")[0])*60 + int(end2.split(":")[1]))
561.                         startInt = end2Int - self.aproxTime
562.                         start2 = str('{:02d}:{:02d}'.format( int(startInt // 60), (startInt -
563. (60 * (startInt // 60))) )

```

```

560.         self.children[3].children[1].text = start2
561.
562.         start = start2
563.
564.         end = str('{:02d}:{:02d}').format( int(start.split(":")[0]) +
        (self.aproxTime // 60), int(start.split(":")[1]) + (self.aproxTime - (60 *
        (self.aproxTime // 60))) )
565.
566.         self.children[1].children[1].text = end
567.
568.         self.children[0].disabled = False
569.
570.     def StartRecipie(self):
571.         self.parent.LoadTheTimer(self.startTimestr, self.endTimestr)
572.
573.
574.     class TheTimer(ScrollView):
575.         def __init__(self, networkDataAsStr, startTimestr, endTimestr, **kwargs):
576.             super().__init__(**kwargs)
577.
578.             from json import loads
579.             networkData = loads(networkDataAsStr)
580.
581.             tmp = BoxLayout(size_hint=(1, None), height="50dp")
582.
583.             print(networkData)
584.
585.             for i in networkData["criticalPath"]:
586.                 buttonWidth = 20 *
                    (networkData["earlyLateTimes"][networkData["network"][i][1]][0] -
                    networkData["earlyLateTimes"][networkData["network"][i][0]][0])
587.
588.                 if buttonWidth == 0:
589.                     continue
590.
591.                 btn = Button(text= i, size_hint= (None, 1), width=
                    "{0}dp".format(buttonWidth))
592.
593.                 tmp.add_widget(btn)
594.
595.                 self.children[0].add_widget(tmp)
596.
597.     class ExitTimerInterfaceButton(ButtonBehavior, Image):
598.         def __init__(self, **kwargs):
599.             super().__init__(**kwargs)
600.
601.             self.source = "media/backArrow.png"
602.             self.on_press=self.ToHomeScreen
603.             self.size_hint= (0.1, None)
604.             self.height=dp(50)
605.
606.         def ToHomeScreen(self):
607.             app= App.get_running_app()
608.             app.root.children[0].LoadMenu(0)
609.
610.
611. if __name__ == '__main__':
612.     CookingTimerApp().run()

```

613.

userManager.py

```

1.
2. import hashlib
3.
4. import databaseManager
5.
6. #sends username and password to server if correct userdata is recieved, this doesn't
   actually let you access anything
7. #you can't as a guest but the client then knows the password you have is right which is
   later used to do things you
8. #need to be logged in to do (like making a recipe)
9. def Login(username, password):
10.     #would be a call to the server
11.     userData = databaseManager.UserFromUsernamePass(username, HashPassword(password))
12.
13.     if userData == None: return -1
14.
15.     print("userManager.py/line 7 --- LOGIN SUCCESSFUL")
16.     return userData
17.
18.
19. def SignUp(username, password):
20.     if IsUsernameAvailable(username) == False:
21.         return False
22.
23.     return databaseManager.AddUser(username, HashPassword(password))
24.
25.
26. #asks the db manager if a username is used
27. def IsUsernameAvailable(username):
28.     if databaseManager.UserFromUsername(username) == None:
29.         return True
30.     return False
31.
32.
33. #has a password, the un-hashed password never leaves the client
34. def HashPassword(password):
35.     return hashlib.sha256(password.encode()).hexdigest()
36.
37.
38. #returns bool, if user had liked a recipe
39. def HasUserLikedRecipe(userID, password, recipeID):
40.     if databaseManager.HasUserLikedRecipe(userID, HashPassword(password), recipeID)
       == "1":
41.         return True
42.     return False
43.
44.
45. #likes/dislikes a recipe
46. def UserLikeRecipeToggle(userID, password, recipeID):
47.     liking = not HasUserLikedRecipe(userID, password, recipeID)
48.
49.     databaseManager.UserLikeRecipeSet(userID, HashPassword(password), recipeID,
       liking)

```

50.

databaseManager.py

```

1.
2. import sqlite3
3.
4. #This would run on the server
5.
6. #(public use) means anyone can fully access the function
7. #(private use) means you need a userID and correct password to access some or all of
   the fuction
8.
9. def RunSQLCommand(databaseName, sqlCommandFrame, *args):
10.     # connecting to the database
11.     connection = sqlite3.connect("Databases/" + databaseName)
12.
13.     # cursor
14.     crsr = connection.cursor()
15.
16.     # print statement will execute if there
17.     # are no errors
18.     print("Connected to the database")
19.
20.     sql_command = str(sqlCommandFrame).format(args=args)
21.
22.     res = crsr.execute(sql_command).fetchall()
23.
24.     # close the connection
25.     connection.close()
26.
27.     return res
28.
29.
30. #gets a specific recipie from an (recipieID) (public use)
31. def GetRecipieFromID(recipieID):
32.     return RunSQLCommand("recipies.db", ""SELECT *
33. FROM TBLRecipies
34. WHERE recipieID={args[0]};""", recipieID)[0]
35.
36.
37. #gets a userRecipie field from (recipieID) (public use)
38. def GetUserRecipieFromID(recipieID):
39.     return RunSQLCommand("recipies.db", ""SELECT *
40. FROM TBLUserRecipies
41. WHERE TBLUserRecipies.recipieID = '{args[0]}'"", recipieID)[0]
42.
43.
44. #gets all recipies (owned/not owned) (liked/not liked) associated with a (userID)
   (public use)
45. def GetUsersRecipiesFromID(userID, owned, liked):
46.     ownedArg = ""
47.     likeArg = ""
48.     if owned != None:
49.         ownedAsStr = "1" if owned else "0"
50.         ownedArg = str("AND TBLUserRecipies.owner = '{0}'".format(ownedAsStr))
51.     if liked != None:

```

```

52.         likedAsStr = "1" if liked else "0"
53.         likeArg = str("AND TBLUserRecipies.liked = '{0}'".format(likedAsStr))
54.
55.         connection = sqlite3.connect("Databases/" + "recipies.db")
56.         # cursor
57.         crsr = connection.cursor()
58.         # print statement will execute if there
59.         # are no errors
60.         print("Connected to the database")
61.
62.         sql_command = """SELECT TBLUserRecipies.userRecipieID, TBLRecipies.*
63. FROM TBLUserRecipies, TBLRecipies
64. WHERE TBLUserRecipies.userID = '{0}' {1} {2} AND TBLUserRecipies.recipieID =
        TBLRecipies.recipieID""".format(userID, ownedArg, likeArg)
65.
66.         res = crsr.execute(sql_command).fetchall()
67.
68.         # close the connection
69.         connection.close()
70.
71.         return res
72.
73.
74. #gets number of reccords in a table of (tableID) (public use)
75. def RecNumbInTable(tableID):
76.     return RunSQLCommand("recipies.db", """SELECT COUNT()
77. FROM {args[0]}""", tableID)[0][0]
78.
79.
80. #adds a recipe and userRecipie reccord to db under (userID) if (password) is correct
    with the other args as infomation
81. def AddRecipie(name, desc, aproxTime, imgID, jsonData, username, password):
82.
83.     if UserFromUsernamePass(username, password) == None: return -1
84.
85.     # connecting to the database
86.     connection = sqlite3.connect("Databases/" + "recipies.db")
87.
88.     # cursor
89.     crsr = connection.cursor()
90.
91.     # print statement will execute if there
92.     # are no errors
93.     print("Connected to the database")
94.
95.     if desc == None: desc = ""
96.     if imgID == None: imgID = ""
97.
98.     sql_command = str("""INSERT INTO TBLRecipies(Name, Description, AproxTime,
        ImageType, NetworkData)
99. VALUES ('{0}', '{1}', '{2}', '{3}', '{4}');""").format(name, desc, aproxTime, imgID,
        jsonData)
100.         crsr.execute(sql_command)
101.
102.         userID = UserFromUsername(username)[0]
103.
104.         sql_command = str("""INSERT INTO TBLUserRecipies(userID, recipieID, owner,
        liked)

```

```

105.     VALUES ('{0}', '{1}', "1", "0");""").format(userID, crsr.lastrowid)
106.         crsr.execute(sql_command)
107.
108.         connection.commit()
109.
110.         # close the connection
111.         connection.close()
112.         return 1
113.
114.
115.     #has a user liked a recipie
116.     def HasUserLikedRecipie(userID, password, recipieID):
117.         username = UserFromUserID(userID)[1]
118.         if UserFromUsernamePass(username, password) == None: return -1
119.
120.         tmp = RunSQLCommand("recipies.db", ""SELECT *
121. FROM TBLUserRecipies
122. WHERE userID = '{args[0]}' AND recipieID = '{args[1]}'""", userID, recipieID)
123.
124.         if tmp == []:
125.             return "0"
126.
127.         return str(tmp[0][4])
128.
129.
130.     #likes/dislikes a recipie
131.     def UserLikeRecipieSet(userID, password, recipieID, likeing):
132.         username = UserFromUserID(userID)[1]
133.
134.         if UserFromUsernamePass(username, password) == None: return -1
135.
136.         # connecting to the database
137.         connection = sqlite3.connect("Databases/" + "recipies.db")
138.
139.         # cursor
140.         crsr = connection.cursor()
141.
142.         # print statement will execute if there
143.         # are no errors
144.         print("Connected to the database")
145.
146.         sql_command = str("""SELECT *
147. FROM TBLUserRecipies
148. WHERE userID = '{0}' AND recipieID = '{1}'""").format(userID, recipieID)
149.         crsr.execute(sql_command)
150.
151.         res = crsr.execute(sql_command).fetchall()
152.
153.         likeingAsTxt = "1" if likeing else "0"
154.
155.         if res == []:
156.             sql_command = str("""INSERT INTO TBLUserRecipies(userID, recipieID, owner,
liked)
157. VALUES ('{0}', '{1}', "0", '{2}');""").format(userID, recipieID, likeingAsTxt)
158.             crsr.execute(sql_command)
159.         else:
160.             sql_command = str("""UPDATE TBLUserRecipies
161. SET liked = '{0}'

```



```

162.     WHERE userID = '{1}' AND recipieID = '{2}';""").format(likeingAsTxt, userID,
    recipieID)
163.         crsr.execute(sql_command)
164.
165.         connection.commit()
166.
167.         # close the connection
168.         connection.close()
169.         return 1
170.
171.
172.     #gets user info, exept password, from (username) (public use)
173.     def UserFromUsername(username):
174.         tmp = RunSQLCommand("recipies.db", ""SELECT userID, username, description
175.         FROM TBLUsers
176.         WHERE username = '{args[0]}'""", username)
177.         if tmp == []:
178.             return None
179.         return tmp[0]
180.
181.
182.     #gets user info, exept password, from (userID) (public use)
183.     def UserFromUserID(userID):
184.         tmp = RunSQLCommand("recipies.db", ""SELECT userID, username, description
185.         FROM TBLUsers
186.         WHERE userID = '{args[0]}'""", userID)
187.
188.         if tmp == []:
189.             return None
190.         return tmp[0]
191.
192.
193.     #checks (password) and then returns user info of (username) if (password) correct
    (private use)
194.     def UserFromUsernamePass(username, password):
195.         tmp = RunSQLCommand("recipies.db", ""SELECT password
196.         FROM TBLUsers
197.         WHERE username = '{args[0]}'""", username)
198.         if tmp == None or tmp == []: return None
199.
200.         if password == tmp[0][0]:
201.             del tmp
202.             return UserFromUsername(username)
203.         return None
204.
205.
206.     #adds a user to the database of (userID) and (password)
207.     def AddUser(username, password):
208.         #does user exist , if exists then don't add user
209.         if UserFromUsername(username) != None: return False
210.
211.         # connecting to the database
212.         connection = sqlite3.connect("Databases/" + "recipies.db")
213.
214.         # cursor
215.         crsr = connection.cursor()
216.
217.         # print statement will execute if there

```

```

218.         # are no errors
219.         print("Connected to the database")
220.
221.         sql_command = str("""INSERT INTO TBLUsers(username, password)
222. VALUES ('{0}', '{1}');""").format(username, password)
223.         crsr.execute(sql_command)
224.
225.         connection.commit()
226.
227.         # close the connection
228.         connection.close()
229.         return True
230.

```

utils.py

```

1.
2. def randomNumbSeq(quantity, startNumb, endNumb, repeated, currentList=[]):
3.     from random import randint
4.     if (not repeated) and ((endNumb-startNumb+1) < quantity):
5.         print("randomNumbSeq argument error - repeated values turned off and quantity
6.         greater than integers in range")
7.         return -1
8.     if (currentList != []):
9.         #cry
10.        pass
11.        numbSeq = currentList
12.        for i in range(quantity):
13.            while (((tempNumb := randint(startNumb, endNumb)) in numbSeq) & (not
14.            repeated)):
15.                pass
16.                numbSeq.append(tempNumb)
17.        return numbSeq
18.
19. def CalculateDependencyTrees(processes):
20.     #make sure there at least one starting proces
21.     startingProcesses = []
22.     for i in processes:
23.         if i.get("conditions") == "":
24.             startingProcesses.append(i.get("id"))
25.
26.     if len(startingProcesses) < 1:
27.         return -1, "Invalid recipie, You need at least one starting process"
28.
29.     #get list of dependencys
30.     dependencys = {}
31.     for i in processes:
32.         tmp = i.get("conditions").split('\n')
33.         dependencys[i.get("id")] = list(filter(lambda a: a != '', tmp))#remove ''
34.         values
35.
36.     startingNodes = list(filter(lambda a: a != None, [(i if dependencys[i]==[] else
37.     None) for i in dependencys]))
38.
39.     #get dependencys backwards
40.     backDependencys = {}#{ "start" : startingNodes}

```

```

38.     for i in dependencys.keys():
39.         tmp = []
40.         for j in dependencys.keys():
41.             if str(i) in dependencys[j]:
42.                 tmp.append(j)
43.
44.         backDependencys[i] = tmp
45.
46.     allNodesVisited = []
47.
48.     #make sure there is no circluar dependencys
49.     for i in startingNodes:
50.         tmp, nodesVisited = TraverseNodeCircularDependencys(i, backDependencys, [], [])
51.
52.         if tmp == False:
53.             return -1, "Invalid recipie, There are circular dependencys"
54.
55.         allNodesVisited += nodesVisited
56.
57.     allNodesVisited = list(set(allNodesVisited))
58.     allNodesVisited.sort()
59.
60.     tmp = list(dependencys.keys())
61.     tmp.sort()
62.
63.     if allNodesVisited != tmp:
64.         return -1, "Invalid recipie, There are circular dependencys"
65.
66.     #calulate ending nodes
67.     endingNodes = list(filter(lambda a: a != None, [(i if backDependencys[i]==[] else
None) for i in backDependencys]))
68.
69.     dependencys['start'] = []
70.     backDependencys['start'] = []
71.
72.     for i in startingNodes:
73.         dependencys[i].append('start')
74.         backDependencys['start'].append(i)
75.
76.     dependencys['end'] = []
77.     backDependencys['end'] = []
78.
79.     for i in endingNodes:
80.         dependencys['end'].append(i)
81.         backDependencys[i].append('end')
82.
83.     return [dependencys, backDependencys, startingNodes, endingNodes], "Valid recipie"
84.
85.
86. def TraverseNodeCircularDependencys(currentPoint, tree, pointsToAvoid, pointsVisited):
87.     pointsVisited.append(currentPoint)
88.     if tree[currentPoint] == []:
89.         return True, pointsVisited
90.
91.     if currentPoint in pointsToAvoid:
92.         return False, None
93.
94.     pointsToAvoid.append(currentPoint)

```

```

95.
96.     for i in tree[currentPoint]:
97.         tmp, foo = TraverseNodeCircularDependencies(i, tree, pointsToAvoid[:],
98.             pointsVisited)
99.         if tmp == False:
100.             return False, None
101.
102.         return True, pointsVisited
103.
104.     def CalculateTimeingNetwork(dependencyTrees, proccessData):
105.         from json import dumps
106.         dependencycs, backDependencycs, startingProcesses, endingProcesses =
107.             dependencyTrees
108.
109.         #find active processes
110.         activeProcesses = list(filter(lambda a: a != None, [ (i.get("id") if
111.             (i.get("activePassive")=="1") else None) for i in proccessData ] ))
112.
113.         #calculate project network (edges are processes)
114.         network, nodes = TraverseProcessesBuildNetwork(startingProcesses,
115.             endingProcesses, dependencycs, backDependencycs)
116.
117.         tmp = network['end'][1]
118.         earlyLateTimes = TraverseNetworkEarlyLateTimes(network, nodes, dependencycs,
119.             backDependencycs, proccessData, {'0': [0, 0], tmp:[0, 0]})
120.
121.         criticalPath = list(reversed(TraverseNetworkCalculateCriticalPath(network,
122.             nodes, earlyLateTimes, ['end'], proccessData)))
123.         aproxTime = earlyLateTimes[network['end'][1]][0]
124.
125.         rtn = {"network" : network, "nodes": nodes, "criticalPath": criticalPath,
126.             "earlyLateTimes": earlyLateTimes, "activeProcesses": activeProcesses}
127.
128.         jsonRtn = dumps(rtn)
129.
130.         return jsonRtn, aproxTime
131.
132.     def TraverseProcessesBuildNetwork(startingNodes, endingNodes, dependencycs,
133.         backDependencycs):
134.         network = {}          # dictionary {"processID"/"dummyID": [startPoint,
135.             endPoint]}
136.         nodes = {}           # dictionary {"ID": ["conditionIn", "conditionIn", ...]}
137.         #conditions are the process that must be compleated for it to start, ignore
138.         dummies just say which process
139.
140.         currentNode = 1      # this is used to keep track of which processID s have
141.             been used
142.         currentDummy = 0     # this is used to keep track of which dummyID s have been
143.             used
144.
145.         #nodes - points in the network
146.         #edges - processes or dummies in the network
147.
148.         processing = True    # used to keep the loop running until the network has been
149.             generated

```

```

140.
141.     cycle = -1          # keeps track of cycle number
142.     lastCycle = ""      # keeps track of last cycles networkdata (to check if
                            network has finished building)
143.
144.     while proccessing:
145.         cycle += 1
146.
147.         for i in dependencys:
148.             #ignore if already added
149.             if network.get(i) ≠ None: continue
150.
151.             #can this edge be added yet (has its dependencys been added)
152.             vailidEdge = True
153.             for j in dependencys[i]:
154.                 if network.get(j) = None:
155.                     vailidEdge = False
156.                     break
157.
158.             #add edge
159.             if vailidEdge:
160.                 #are its dependencys going to same node?
161.                 tmp = True
162.                 firstDepend = '-1'
163.                 if len(dependencys[i]) > 0: firstDepend = dependencys[i][0]
164.                 for j in dependencys[i]:
165.                     if network.get(j)[1] ≠ network.get(firstDepend)[1]:
166.                         tmp = False
167.
168.                 #Y - add edge from that node
169.                 if tmp:
170.                     foo = network.get(firstDepend)
171.                     startNode = foo[1] if foo ≠ None else "0"
172.                     network[i] = [str(startNode), str(currentNode)]
173.                     if nodes.get(currentNode) = None: nodes[str(currentNode)] =
174. [i]
175.                     else: nodes[currentNode].append(i)
176.                     currentNode += 1
177.
178.                 #N - continue
179.                 else:
180.                     #does one/more of its dependencys act as a dependency for
something else?
181.                     sharedDependencys = []
182.
183.                     for currentProccessDependency in dependencys[i]:
184.                         if len(backDependencys[currentProccessDependency]) > 1:
185.                             sharedDependencys.append(currentProccessDependency)
186.
187.                     if len(sharedDependencys) ≠ 0:
188.                         #Y
189.                         #make listA (list of something elses that share the same
set of dependencys)
190.                         listA = []
191.                         #make listB (list of something elses that DON'T share the
same set of dependencys)
192.                         listB = []
193.                         listB_B = [] #list of what they share

```

```

193.
194.         #get list of something elses somethingElseID
195.         somethingElses = []
196.         for j in sharedDependencycs:
197.             tmp = []
198.             for k in backDependencycs[j]:
199.                 if k not in somethingElses and k  $\neq$  i:
200.                     somethingElses.append(k)
201.
202.         #for each something else:
203.         for j in somethingElses:
204.             #does that something else share the same set of
dependencies?
205.             if sorted(dependencycs[j]) == sorted(dependencycs[i]):
206.                 #Y - Add to listA
207.                 listA.append(j)
208.             else:
209.                 #N - Add to listB
210.                 listB.append(j)
211.                 for q in dependencycs[j]:
212.                     if q in dependencycs[i]:
213.                         listB_B.append(q)#add what j shares with i
214.                         continue
215.
216.         #create temporary listB, make empty (for dummy nodes)
217.         tmpDummy = {}
218.         #{"dummyID" : startNode}
219.
220.         #for each item in listB:
221.         for j in range(0, len(listB)):
222.             print("H")
223.             #add a dummy to tmpDummy
224.             nodeOrigin = network.get(j)
225.
226.             if nodeOrigin == None:
227.                 nodeOrigin = listB_B[j]
228.
229.             for k in backDependencycs:
230.                 if listB[j] in backDependencycs[k]:
231.                     nodeOrigin = network[k][1]
232.
233.             tmpDummy["dummy" + str(currentDummy)] = nodeOrigin
234.             currentDummy += 1
235.
236.         #combine dependencycs in temporary node to real node
237.         nodeID = ''
238.
239.         print(dependencycs[i])
240.
241.         for j in dependencycs[i]:
242.             if j not in sharedDependencycs:
243.                 nodeID = network.get(j)[1]
244.                 break
245.
246.         if nodeID == '':
247.             nodeID = currentNode
248.             currentNode += 1
249.

```

```

250.
251.         nodes[nodeID] = list(set(dependencys[i]))
252.         #add dummies from dependencys to real node (tmpDummy)
253.         for j in tmpDummy:
254.             network[j] = [tmpDummy[j], str(nodeID)]
255.             nodes[nodeID].append(j)
256.         #combine other dependencys (remove old nodes)
257.         for j in dependencys[i]:
258.             if j not in sharedDependencys and (network[j][1] ≠
nodeID):
259.                 nodes.pop(str(network[j][1]))
260.                 network[j][1] = nodeID
261.             #add the new edge and node
262.             network[i] = [str(nodeID), str(currentNode)]
263.             nodes[str(currentNode)] = [i]
264.             currentNode += 1
265.
266.         else:
267.             #N - combine dependencys to same node then add edge to node
(update node info)
268.
269.             nodesToCombine = [network[j][1] for j in dependencys[i]]
270.
271.             newNodeInfo = []
272.             for j in nodesToCombine:
273.                 for k in nodes[j]:
274.                     newNodeInfo.append(k)
275.
276.             for j in nodesToCombine[1:]:
277.                 nodes.pop(j)
278.
279.             nodes[nodesToCombine[0]] = newNodeInfo
280.
281.             for j in newNodeInfo:
282.                 network[j][1] = str(nodesToCombine[0])
283.
284.             network[i] = [str(nodesToCombine[0]), str(currentNode)]
285.             if nodes.get(currentNode) == None: nodes[str(currentNode)]
= [i]
286.             else: nodes[currentNode].append(i)
287.             currentNode += 1
288.
289.
290.         #if all dependencys in network
291.         #Y - proccessing = False
292.
293.         thisCycle = str([network, nodes])
294.         if thisCycle == lastCycle:
295.             proccessing = False
296.
297.         lastCycle = str([network, nodes])
298.
299.         return network, nodes
300.
301.
302.     def TPBN_WhichNodeHasIn(nodes, id):
303.         for i in nodes:
304.             if id in nodes[i]:

```

```

305.         return i
306.     return -1
307.
308.
309.     def TPBN_WhichNodeHasOut(network, id):
310.         for i in network:
311.             if id == network[i][0]:
312.                 return i
313.         return -1
314.
315.
316.     def TraverseNetworkEarlyLateTimes(network, nodes, dependencycs, backDependencycs,
317.         proccessData, earlyLateTimes):
318.         TNELT_EarlyTimes("start", network, nodes, backDependencycs, proccessData,
319.             earlyLateTimes)
320.
321.         finalNode = '0'
322.         totalTime = 0
323.         for i in earlyLateTimes:
324.             if earlyLateTimes[i][0] > totalTime:
325.                 totalTime = earlyLateTimes[i][0]
326.                 finalNode = i
327.
328.         earlyLateTimes[finalNode][1] = earlyLateTimes[finalNode][0]
329.
330.         TNELT_LateTimes("end", network, nodes, dependencycs, proccessData,
331.             earlyLateTimes)
332.
333.         return earlyLateTimes
334.
335.     def TNELT_EarlyTimes(currentEdge, network, nodes, backDependencycs, proccessData,
336.         earlyLateTimes):
337.         currentProccessTime = None
338.         if (currentEdge == 'start' or currentEdge == "end" or ("dummy" in
339.             currentEdge)):
340.             currentProccessTime = 0
341.         else:
342.             currentProccessTime = int(FindProccesInfoFromID(currentEdge,
343.                 proccessData)['aproxTime'])
344.
345.         if (earlyLateTimes.get(network[currentEdge][1]) == None):
346.             earlyLateTimes[network[currentEdge][1]] =
347.                 [earlyLateTimes.get(network[currentEdge][0])[0] + currentProccessTime, None]
348.
349.         if (earlyLateTimes.get(network[currentEdge][1])[0] <
350.             earlyLateTimes[network[currentEdge][0]][0] + currentProccessTime):
351.             earlyLateTimes[network[currentEdge][1]][0] =
352.                 earlyLateTimes.get(network[currentEdge][0])[0] + currentProccessTime
353.
354.         tmp = []
355.         for i in network:
356.             if network[i][0] == network[currentEdge][1]:
357.                 tmp.append(i)
358.
359.         for i in tmp:

```



```

353.         TNELT_EarlyTimes(i, network, nodes, backDependency, processData,
        earlyLateTimes)
354.
355.
356.     def TNELT_LateTimes(currentEdge, network, nodes, dependency, processData,
        earlyLateTimes):
357.         if network[currentEdge][0] == '0':
358.             return earlyLateTimes
359.
360.         currentProcessTime = None
361.
362.         if (currentEdge == 'start' or currentEdge == "end" or "dummy" in currentEdge):
363.             currentProcessTime = 0
364.         else:
365.             currentProcessTime = int(FindProcessInfoFromID(currentEdge,
        processData)['approxTime'])
366.
367.             if (earlyLateTimes.get(network[currentEdge][0])[1] == None):
368.                 earlyLateTimes[network[currentEdge][0]][1] =
        earlyLateTimes.get(network[currentEdge][1])[1] - currentProcessTime
369.
370.             if (earlyLateTimes.get(network[currentEdge][0])[1] >
        earlyLateTimes[network[currentEdge][1]][1] - currentProcessTime):
371.                 earlyLateTimes[network[currentEdge][0]][1] =
        earlyLateTimes.get(network[currentEdge][1])[1] - currentProcessTime
372.
373.         if network[currentEdge][0] == '0':
374.             return
375.
376.         tmp = []
377.         for i in network:
378.             if network[i][1] == network[currentEdge][0]:
379.                 tmp.append(i)
380.
381.
382.         for i in tmp:
383.             TNELT_LateTimes(i, network, nodes, dependency, processData,
        earlyLateTimes)
384.
385.
386.     def TraverseNetworkCalculateCriticalPath(network, nodes, earlyLateTimes,
        criticalPath, processData):
387.         if network[criticalPath[-1]][0] == '0':
388.             return criticalPath
389.
390.         currentProcessTime = None
391.         if (criticalPath[-1] == 'start' or criticalPath[-1] == "end" or "dummy" in
        criticalPath[-1]):
392.             currentProcessTime = 0
393.         else:
394.             currentProcessTime = int(FindProcessInfoFromID(criticalPath[-1],
        processData)['approxTime'])
395.
396.         for i in nodes[network[criticalPath[-1]][0]]:
397.             if earlyLateTimes[network[i][1]][0] == earlyLateTimes[network[i][1]][1] and
        (earlyLateTimes[network[i][1]][0] == earlyLateTimes[network[criticalPath[-1]][1]][0] -
        currentProcessTime):
398.                 criticalPath.append(i)

```

```

399.         break
400.
401.     #error in TNELT_EarlyTimes/TNELT_LateTimes - None values as times?
402.     print(criticalPath[-1])
403.     print(network)
404.     print(earlyLateTimes)
405.
406.     from time import sleep
407.     sleep(0.1)
408.
409.     TraverseNetworkCalculateCriticalPath(network, nodes, earlyLateTimes,
        criticalPath, proccessData)
410.
411.     return criticalPath
412.
413. def FindProccesInfoFromID(id, proccessData):
414.     for i in proccessData:
415.         if i['id'] == id:
416.             return i
417.     return ['error']
418.

```

XML

```

1.
2.  #:kivy 2.2.0
3.
4. <CookingTimerInterface>:
5.     orientation: "vertical"
6.
7.     WelcomeScreenInterface:
8.
9.
10. <WelcomeScreenInterface@BoxLayout>:
11.     orientation: "vertical"
12.
13.     LoginInterface:
14.
15.     BoxLayout:
16.         orientation: "horizontal"
17.         size_hint: 1, None
18.         height: "40dp"
19.
20.         Button:
21.             text: "Login"
22.             color: 1, 1, 1, 1
23.             on_press: root.LoadMenu(0)
24.
25.         Button:
26.             text: "Sign Up"
27.             color: 1, 1, 1, 1
28.             on_press: root.LoadMenu(1)
29.
30.
31. <LoginInterface@BoxLayout>:
32.     orientation: "vertical"
33.     AnchorLayout:

```

```
34.         anchor_x: "center"
35.         anchor_y: "center"
36.     BoxLayout:
37.         orientation: "vertical"
38.         size_hint: None, None
39.         height: "120dp"
40.         width: "200dp"
41.
42.     TextInput:
43.         id: usernameInput
44.
45.         size_hint: None, None
46.         size: "200dp", "40dp"
47.
48.         pos_hint: {"center_x" : 0.5}
49.
50.         hint_text: "username"
51.         multiline: False
52.
53.     TextInput:
54.         id: passwordInput
55.
56.         size_hint: None, None
57.         size: "200dp", "40dp"
58.
59.         pos_hint: {"center_x" : 0.5}
60.
61.         hint_text: "password"
62.         multiline: False
63.         password: True
64.
65.     Button:
66.         text: "Login"
67.         color: 1, 1, 1, 1
68.
69.         size_hint: None, None
70.         size: "200dp", "40dp"
71.
72.         pos_hint: {"center_x" : 0.5}
73.
74.         on_press: root.SubmitLoginDetails()
75.
76.
77. <SignupInterface@BoxLayout>:
78.     orientation: "vertical"
79.
80.     AnchorLayout:
81.         anchor_x: "center"
82.         anchor_y: "center"
83.     BoxLayout:
84.         orientation: "vertical"
85.         size_hint: None, None
86.         height: "120dp"
87.         width: "200dp"
88.
89.     TextInput:
90.         id: usernameInput
91.
```

```

92.         size_hint: None, None
93.         size: "200dp", "40dp"
94.
95.         pos_hint: {"center_x" : 0.5}
96.
97.         hint_text: "username"
98.         multiline: False
99.
100.        TextInput:
101.            id: passwordInput
102.
103.            size_hint: None, None
104.            size: "200dp", "40dp"
105.
106.            pos_hint: {"center_x" : 0.5}
107.
108.            hint_text: "password"
109.            multiline: False
110.            password: True
111.
112.        TextInput:
113.            id: passwordInputConfirm
114.
115.            size_hint: None, None
116.            size: "200dp", "40dp"
117.
118.            pos_hint: {"center_x" : 0.5}
119.
120.            hint_text: "confirm password"
121.            multiline: False
122.            password: True
123.
124.        Button:
125.            text: "Sign Up"
126.            color: 1, 1, 1, 1
127.
128.            size_hint: None, None
129.            size: "200dp", "40dp"
130.
131.            pos_hint: {"center_x" : 0.5}
132.
133.            on_press: root.SubmitSignUpDetails()
134.
135.        Label:
136.            size_hint: 1, None
137.            height: "40dp"
138.            text: root.signinFeedbackText
139.
140.
141.    <LoggedInInterface@BoxLayout>:
142.        orientation: "vertical"
143.
144.        BoxLayout:
145.            orientation: "horizontal"
146.            size_hint: 1, None
147.            height: "40dp"
148.
149.            Button:

```

```

150.         text: "Recipies"
151.         on_press: root.LoadMenu(0)
152.
153.         Button:
154.             text: "New Recipie"
155.             on_press: root.LoadMenu(1)
156.
157.         Button:
158.             text: "My Profile"
159.             on_press: root.LoadMenu(2, -1)
160.
161.         RecipeSearchInterface:
162.
163.
164.         <RecipeSearchInterface@BoxLayout>:
165.             orientation: "vertical"
166.
167.         RecipeSearchScrollInterface:
168.
169.
170.         <RecipeSearchScrollInterface@ScrollView>:
171.
172.
173.         <RecipeSearchBoxInterface>:
174.             orientation: "vertical"
175.             spacing: "5dp"
176.
177.             #         left top right bottom
178.             padding: ("10dp", "5dp", "10dp", 0)
179.             size_hint: 1, None
180.             height: self.minimum_height
181.
182.
183.         <RecipeWigitBanner@BoxLayout>:
184.             orientation: "horizontal"
185.             size_hint: 1, None
186.             height: "100dp"
187.             spacing: "5dp"
188.
189.         canvas:
190.             Color:
191.                 rgb: (0.1, 0.1, 0.12)
192.             Rectangle:
193.                 pos: self.pos
194.                 size: self.size
195.
196.         on_press: root.LoadRecipePage()
197.
198.
199.         <RecipeViewInterface@BoxLayout>:
200.             orientation: "vertical"
201.
202.         BoxLayout:
203.             orientation: "horizontal"
204.             size_hint: 1, None
205.             height: "40dp"
206.
207.         Button:

```

```

208.         text: "Basic Info"
209.         on_press: root.LoadRecipieViewPage(0)
210.
211.     Button:
212.         text: "COOK IT!"
213.         on_press: root.LoadRecipieViewPage(1)
214.
215.
216.     Button:
217.         text: "Nutrition"
218.         on_press: root.LoadRecipieViewPage(2)
219.
220.
221. <RecipieViewInterfaceBasicInfo@BoxLayout>:
222.     orientation: "vertical"
223.
224.
225.
226. <NewRecipieInterface@BoxLayout>:
227.     orientation: "vertical"
228.
229.     BoxLayout:
230.         orientation: "horizontal"
231.         size_hint: 1, None
232.         height: "40dp"
233.
234.         Button:
235.             text: "Basic Info"
236.             on_press: root.LoadRecipieMakingPage(0)
237.
238.         Button:
239.             text: "Timings"
240.             on_press: root.LoadRecipieMakingPage(1)
241.
242.         Button:
243.             text: "Finish"
244.             on_press: root.LoadRecipieMakingPage(2)
245.
246.
247. <NewRecipieBasicInfo@BoxLayout>:
248.     orientation: "vertical"
249.     AnchorLayout:
250.         anchor_x: "center"
251.         anchor_y: "center"
252.         BoxLayout:
253.             orientation: "vertical"
254.             size_hint: None, None
255.             height: "120dp"
256.             width: "200dp"
257.
258.             TextInput:
259.                 size_hint: None, None
260.                 size: "200dp", "40dp"
261.
262.                 pos_hint: {"center_x" : 0.5}
263.
264.                 hint_text: "Recipie Name"
265.                 multiline: False

```

```
266.
267.         TextInput:
268.             size_hint: None, None
269.             size: "200dp", "150dp"
270.
271.             pos_hint: {"center_x" : 0.5}
272.
273.             hint_text: "Description"
274.             multiline: True
275.
276.
277. <NewRecipieTimingInfo@BoxLayout>:
278.     orientation: "vertical"
279.
280.     ScrollView:
281.         NewRecipieProccessList:
282.
283.
284.
285. <NewRecipieProccessList@BoxLayout>:
286.     orientation: "vertical"
287.
288.     spacing: "10dp"
289.
290.     size_hint: 1, None
291.     size: self.minimum_size
292.
293.     Button:
294.         on_press: root.NewProccess()
295.         text: "NewProccess"
296.
297.         size_hint: 1, None
298.         height: "50dp"
299.
300.
301. <NewRecipieProcessBox@BoxLayout>:
302.     size_hint: 1, None
303.     height: "325dp"
304.
305.     padding: ("5dp", 0, "5dp", 0)
306.
307.     orientation: "vertical"
308.
309.     canvas:
310.         Color:
311.             rgb: (0.1, 0.1, 0.12)
312.         Rectangle:
313.             pos: self.pos
314.             size: self.size
315.
316.     Label:
317.         size_hint: None, None
318.         size: "30dp", "25dp"
319.
320.         text: "ID"
321.
322.     ToggleButton:
323.         size_hint: 1, None
```

```

324.         height: "40dp"
325.
326.         text: "Active Proccess" if self.state=="down" else "Passive Proccess"
327.
328.     TextInput:
329.         size_hint: 1, None
330.         height: "40dp"
331.
332.         hint_text: "Aprox time (in minutes)"
333.         multiline: False
334.
335.     TextInput:
336.         size_hint: 1, None
337.         height: "100dp"
338.
339.         hint_text: "IDs of proccess which need to finish before this starts
(seperate by new lines) or leave blank if none (only one can be left blank and it will
be the first thing in the recipie)"
340.         multiline: True
341.
342.     TextInput:
343.         size_hint: 1, None
344.         height: "100dp"
345.
346.         hint_text: "Description of this proccess"
347.         multiline: True
348.
349.
350. <NewRecipieConfirm@BoxLayout>:
351.     orientation: "vertical"
352.
353.     Button:
354.         text: "Check recipie"
355.         size_hint: None, None
356.         size: "50dp", "50dp"
357.         on_press: root.CheckRecipie()
358.
359.     Label:
360.         text: ""
361.
362.     Button:
363.         text: "Create recipie!"
364.         size_hint: None, None
365.         size: "50dp", "50dp"
366.         disabled: True
367.         on_press: root.CreateRecipie()
368.
369.
370. <TimerInterface@BoxLayout>:
371.     orientation: "vertical"
372.
373.
374. <TheTimer@ScrolllView>:
375.     do_scroll_x: True
376.     do_scroll_y: True
377.
378.     size_hint: 1, 0.4
379.     pos_hint: {'y': 0}

```



```

380.
381.
382.     BoxLayout:
383.         orientation: "vertical"
384.         size_hint: None, None
385.         height: self.minimum_height
386.         width: "2000dp"
387.
388.         canvas.after:
389.             Rectangle:
390.                 pos: 40, 100
391.                 size: 3, 200
392.
393.
394.     <TimerSetup@BoxLayout>:
395.         orientation: "vertical"
396.
397.         Label:
398.             text: "Start at :"
399.
400.             size_hint: 0.9, None
401.             height: "50dp"
402.
403.             pos_hint: {"center_x" : 0.5}
404.
405.         BoxLayout:
406.             size_hint: 0.9, None
407.             pos_hint: {"center_x" : 0.5}
408.
409.             TextInput:
410.                 hint_text: "Time when you want to start at (aproximate) (HH:MM)"
411.
412.                 size_hint: 0.7, None
413.                 height: "50dp"
414.
415.                 pos_hint: {"center_x" : 0.5}
416.
417.             Button:
418.                 text: "Submit"
419.
420.                 size_hint: None, None
421.                 size: "70dp", "50dp"
422.
423.                 on_press: root.OnTextChange(0)
424.
425.         Label:
426.             text: "End at : +" + root.aproxTimeStr
427.
428.             size_hint: 0.9, None
429.             height: "50dp"
430.
431.             pos_hint: {"center_x" : 0.5}
432.
433.         BoxLayout:
434.             size_hint: 0.9, None
435.             pos_hint: {"center_x" : 0.5}
436.
437.         TextInput:

```

```

438.         hint_text: "Time when you want to finish by (aproximate) (HH:MM)"
439.
440.         size_hint: 0.7, None
441.         height: "50dp"
442.
443.         Button:
444.             text: "Submit"
445.
446.             size_hint: None, None
447.             size: "70dp", "50dp"
448.
449.             on_press: root.OnTextChange(1)
450.
451.         Button:
452.             text: "Create recipie!"
453.             size_hint: None, None
454.             size: "50dp", "50dp"
455.             disabled: True
456.             on_press: root.StartRecipie()
457.
458.
459.     <ProfileViewInterface@BoxLayout>:
460.         orientation: "vertical"
461.
462.         Label:
463.             text: "Profile"
464.             size_hint: 1, 0.1
465.

```

SQL

```

BEGIN TRANSACTION;
CREATE TABLE IF NOT EXISTS "TBLUsers" (
    "userID"    INTEGER NOT NULL UNIQUE,
    "username"  TEXT NOT NULL UNIQUE,
    "password"  TEXT NOT NULL,
    "description" TEXT,
    PRIMARY KEY("userID" AUTOINCREMENT)
);
CREATE TABLE IF NOT EXISTS "TBLIngredients" (
    "ingredientID" INTEGER NOT NULL UNIQUE,
    "name" TEXT NOT NULL,
    PRIMARY KEY("ingredientID" AUTOINCREMENT)
);
CREATE TABLE IF NOT EXISTS "TBLRecipieIngredients" (
    "recipieIngredientID" INTEGER NOT NULL UNIQUE,
    "recipieID" INTEGER NOT NULL,
    "ingredientID" INTEGER NOT NULL,
    "quantity" INTEGER NOT NULL,
    "units" TEXT,

```

```
PRIMARY KEY("recipeIngredientID" AUTOINCREMENT),
FOREIGN KEY("recipeID") REFERENCES "TBLRecipes"("recipeID"),
FOREIGN KEY("ingredientID") REFERENCES
"TBLIngredients"("ingredientID")
);
CREATE TABLE IF NOT EXISTS "TBLUserRecipes" (
    "userRecipeID" INTEGER NOT NULL UNIQUE,
    "userID" INTEGER NOT NULL,
    "recipeID" INTEGER NOT NULL,
    "owner" INTEGER NOT NULL,
    "liked" INTEGER NOT NULL,
    PRIMARY KEY("userRecipeID" AUTOINCREMENT),
    FOREIGN KEY("userID") REFERENCES "TBLUsers"("userID"),
    FOREIGN KEY("recipeID") REFERENCES "TBLRecipes"("recipeID")
);
CREATE TABLE IF NOT EXISTS "TBLRecipes" (
    "recipeID" INTEGER NOT NULL UNIQUE,
    "Name" TEXT NOT NULL,
    "Description" TEXT,
    "AproxTime" INTEGER NOT NULL,
    "ImageType" TEXT,
    "NetworkData" TEXT NOT NULL,
    PRIMARY KEY("recipeID" AUTOINCREMENT)
);
COMMIT;
```

References:

001 - Samsung (22/02/2022) Whisk [\[whisk.com/recipe-sharing-app\]](https://whisk.com/recipe-sharing-app) accessed(23/03/2023)

002 - Maxim Kabluka (24/01/2023) Kitchen Multi-Timer

[\[play.google.com/store/apps/details?id=com.maxxt.kitchentimer&hl=en_GB&gl=US\]](https://play.google.com/store/apps/details?id=com.maxxt.kitchentimer&hl=en_GB&gl=US)

accessed(23/03/2023)

003 - CobraApps (04/03/2023) Cooking Timer: scheduler

[\[play.google.com/store/apps/details?id=com.cobraapps.cookingtimer&hl=en_GB&gl=US\]](https://play.google.com/store/apps/details?id=com.cobraapps.cookingtimer&hl=en_GB&gl=US)

accessed(23/03/2023)

004 - Statcounter (1/05/2023) Desktop vs Mobile vs Tablet Market Share Worldwide

[\[gs.statcounter.com/platform-market-share/desktop-mobile-tablet\]](https://gs.statcounter.com/platform-market-share/desktop-mobile-tablet) accessed(03/06/2023)

005 - Statcounter (01/03/2023) Mobile Operating System Market Share Worldwide

[\[gs.statcounter.com/os-market-share/mobile/worldwide\]](https://gs.statcounter.com/os-market-share/mobile/worldwide) accessed(29/04/2023)

006 - Statcounter (01/03/2023) Android Version Market Share Worldwide

[\[gs.statcounter.com/os-version-market-share/android\]](https://gs.statcounter.com/os-version-market-share/android) accessed(29/04/2023)

007 - Wikipedia (17/02/2023) Gantt Chart - Wikipedia [\[en.wikipedia.org/wiki/Gantt_chart\]](https://en.wikipedia.org/wiki/Gantt_chart)

accessed(03/06/2023)

008 - Wikipedia (2/06/2023) Program evaluation and review technique - Wikipedia

[\[en.wikipedia.org/wiki/Program_evaluation_and_review_technique\]](https://en.wikipedia.org/wiki/Program_evaluation_and_review_technique) accessed(03/06/2023)

009 - Gusto (01/01/2023) Gusto [\[gousto.co.uk\]](https://gousto.co.uk) accessed(04/06/2023)

010 - Wikipedia (30/10/2023) MySQL [\[en.wikipedia.org/wiki/MySQL\]](https://en.wikipedia.org/wiki/MySQL) accessed (03/11/2023)